

Anwendung von Datenbanksystemen

Prof. Dr.-Ing. Simon Hufnagel

Hochschule für angewandte Wissenschaften Ansbach

16. März 2026

- 1 Einführung
- 2 Entity Relationship Modell
- 3 technisches Modell
- 4 Qualitätskriterien für relationales Modell
- 5 SQL - Übersicht und Data Definition Language
- 6 SQL - Data Definition Language Teil 2
- 7 SQL - Data Manipulation Language

1 Einführung

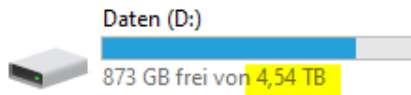
- Motivation
- Aufbau von Datenbanken
- Typen strukturorientierter DBS
- Organisatorisches

„Daten sind der Rohstoff des 21. Jahrhunderts“

Quelle: u.a. Dr. Angela Merkel Bundeskanzlerin der Bundesrepublik Deutschland 2005 - 2021

„Es gibt nicht zu viele Daten“

Quelle: Ben Lorica, Direktor für Inhaltsstrategie bei O'Reilly Media



- Speicherkapazitäten heutiger PCs üblicherweise im Terabyte (TB) Bereich
- 1 TB = 1.000.000.000.000 Byte (1 Billion)
- reicht zum Speichern von ...
 - ... 250.000.000.000 32-Bit Zahlen
 - ... etwa 200.000.000 DIN A4 Seiten Text
 - ... etwa 300.000 Smartphone Fotos¹
 - ... etwa 300 Stunden Full-HD Filmmaterial¹
 - ...

¹abhängig von div. Qualitätseinstellungen...

- **viele** gleichartige „**Objekte**“ mit vielfältigen aber ähnlichen **Eigenschaften**
 - Bücher
 - Menschen
 - Orte
 - Bezahlvorgänge
 - Verträge
 - Zeugnisse
- Unterteilung in **Daten** und **Metadaten**, z.B. bei Fotos:
 - Daten: Farbe der jeweiligen Pixel des Fotos
 - Metadaten: Auflösung, Aufnahmeort, Name des Photographen, verwendetes Kameramodell, etc...
- **Zusammenhänge** innerhalb der Daten, z.B. bei Geschäftsprozessen:
 - Mitarbeitende sind Abteilungen zugeordnet
 - Bestellungen sind Lieferanten zugeordnet
 - Bestellungen enthalten Artikel
 - Bestellungen werden von Mitarbeitenden bearbeitet

Ohne Datenbank:

- Speicherung der Daten auf dem Rechner im **Dateisystem**
- Verwendung „üblicher“ **Dateiformate**
 - Textdokumente
 - Spreadsheets
 - Präsentationen
 - ...

Nachteile

- passendes Ordnungsschema?
- Speicherung von Metadaten eingeschränkt / in separaten Dateien
- eingeschränkte Suchmöglichkeiten
- ggf. redundante Datenhaltung nötig
- keine Sicherstellung der Datenintegrität
- eingeschränkte Möglichkeiten der Zugriffskontrolle



- **Datenspeicherung**

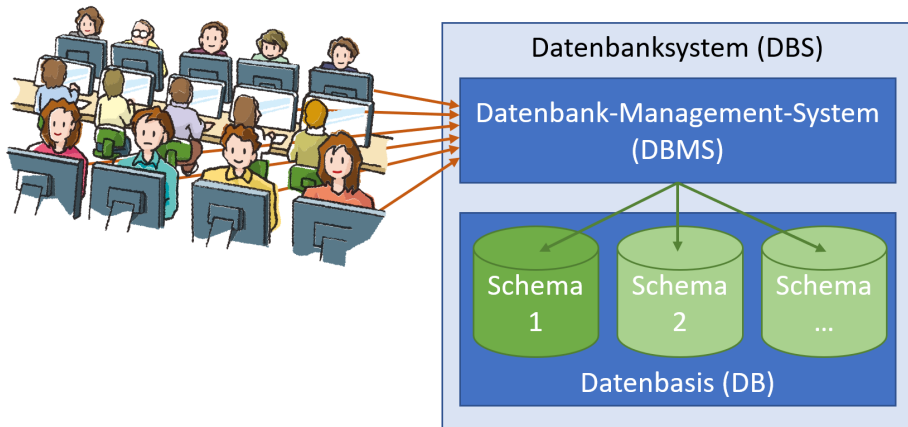
- persistent
- zuverlässig
- zentral
- Sicherstellung der Integrität

- komfortabler und flexibler **Zugriff** auf die Daten

- **Suche** nach beliebigen Kriterien
- hohe **Performanz** für wiederkehrende Abfragen
- gleichzeitiger Zugriff **mehrerer Benutzer**
- Anbindung div. **Programmiersprachen**
- **Ad-hoc** Abfragen

- umfangreiches **Zugriffsrechte** Management

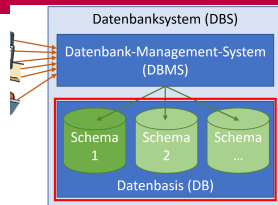
- feingranular
- Einsatz von Rollen



Datenbanksystem = Datenbank-Management-System + Datenbasis

Eigenschaften

- persistente Speicherung
 - Enthält Gesamtheit der Daten des Anwendungsbereichs
 - wichtigste Komponente: **Schema**
 - **mehrere Schemata** pro Datenbasis möglich
 - Begriff „Schema“ wird nicht immer eindeutig verwendet
- mehrere Bedeutungen
- Definition der an die Realität angepassten **Struktur** der Datenablage
 - **Datenmenge** der jeweiligen Anwendung nach vor genannter Struktur



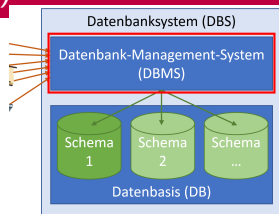
Datenbank-Management-System (DBMS)

Eigenschaften

- besteht aus diversen Verwaltungsprogrammen
- je nach Produkt: sehr große Skalierbarkeit

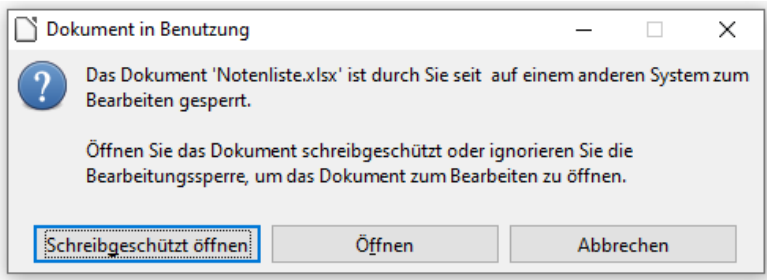
Aufgaben

- Schnittstelle zum Anwender
 - bietet unterschiedliche Zugriffsmöglichkeiten
 - Rechteverwaltung
 - Verhindert direkten Zugriff der Nutzer auf die Datenbasis
 - Management konkurrierender Zugriffe (siehe folgende Folien)
- Verwaltung der Datenbasis
 - Abfrage von Daten
 - Manipulation (Ergänzen, Ändern, Löschen) von Daten
 - Sicherstellung der Integrität
- Administrative Funktionen
 - Backup
 - Protokollierung



Management konkurrierender Zugriffe

- Ziel eines DBMS: Nutzung der gespeicherten Daten durch **mehrere** Nutzer **gleichzeitig**
 - Herausforderung: alle Nutzer müssen **zu jeder Zeit** einen **konsistenten** Datensatz vorfinden
 - **pragmatischer** Ansatz: Sperrung des Zugriffs auf die **komplette Datenbasis** während ein Nutzer die Datenbasis verwendet
- **Serialisierung** der Zugriffe → Nutzer B muss warten, bis der Zugriff von Nutzer A vollständig beendet ist



- stattdessen: sehr **feingranulare Sperrung** der für den **aktuellen Zugriff** eines Nutzers nötigen **Teile** des Datenbestandes
- wenn Nutzer B andere Teile des Datenbestandes benötigt **echte Gleichzeitigkeit** möglich
- wenn sich die benötigten Teil des Datenbestandes überschneiden kann es zu diversen Problemen kommen, die durch das DBMS gelöst werden müssen
 - Lost Update
 - Dirty Read
 - Unrepeatable Read
 - Phantom Read
- Weitere Details dazu in einer späteren Vorlesung.

1 Einführung

- Motivation
- Aufbau von Datenbanken
- Typen strukturorientierter DBS
- Organisatorisches

- bereits bekannt: DBS erlaubt grundsätzlich eine an die Realität anpassbare Struktur der Datenablage
- ABER: je nach DBS existieren **unterschiedliche** Strukturierungsmöglichkeiten
- (historisch) lassen sich folgende **Typen** strukturorientierter DBS unterscheiden
 - **hierarchische** Datenbank
 - **Netzwerk** Datenbank
 - **relationale** Datenbank
 - **objektorientierte** Datenbank

- hierarchische Gliederung der Daten

→ **Baumstruktur**

- gut geeignet für **hierarchisch strukturierte Informationen** (z.B. Organigramm)
- **Vorteil:** sehr einfache Struktur
- **Nachteil:** keine Flexibilität
- heute kaum noch als Datenbank im Einsatz (Ausnahme: klassische Dateisysteme)

- Erweiterung der hierarchischen Datenbank um die Möglichkeit **mehrere Elternelemente** zu verwenden
- keine strenge Hierarchie mehr
- **Vorteil:** hohe Flexibilität bei der Abbildung von Abhängigkeiten zwischen Datenelementen
- **Nachteil:** wird schnell unübersichtlich

- **freie Definition** der Struktur möglich
- fasst Eigenschaften von Objekten zu **Relationen** zusammen
- Beziehungen zwischen Objekten lassen sich über **Schlüssel** abbilden
- **Vorteil:** Anpassung der Datenstruktur an die Anforderungen der Realität und nicht umgekehrt
- **Nachteil:** jede Datenbasis hat eigene Struktur → Einarbeitung notwendig

Popularität relationaler Datenbank-Management-Systemen:

☐ sekundäre Datenbankmodelle berücksichtigen

166 Systeme im Ranking, Februar 2023

Rang			DBMS	Datenbankmodell	Punkte		
Feb 2023	Jan 2023	Feb 2022			Feb 2023	Jan 2023	Feb 2022
1.	1.	1.	Oracle	Relational, Multi-Model	1247,52	+2,35	-9,31
2.	2.	2.	MySQL	Relational, Multi-Model	1195,45	-16,51	-19,23
3.	3.	3.	Microsoft SQL Server	Relational, Multi-Model	929,09	+9,70	-19,96
4.	4.	4.	PostgreSQL	Relational, Multi-Model	616,50	+1,65	+7,12
5.	5.	5.	IBM Db2	Relational, Multi-Model	142,97	-0,60	-19,91
6.	7.	7.	SQLite	Relational	132,67	+1,17	+4,30
7.	6.	6.	Microsoft Access	Relational	131,03	-2,33	-0,23
8.	8.	10.	Snowflake	Relational	115,65	-1,60	+32,47
9.	9.	8.	MariaDB	Relational, Multi-Model	96,81	-2,55	-10,30
10.	10.	9.	Microsoft Azure SQL Database	Relational, Multi-Model	78,75	-1,62	-6,20

Source: <https://db-engines.com/de/ranking/relational+dbms>

- Struktur analog zur **Objektorientierten Programmierung** (Klassen, Vererbung, Objekte als Instanzen von Klassen)
- alle Eigenschaften und Methoden eines beliebig komplexen Objektes werden **gemeinsam** gespeichert und nicht über mehrere Tabellen verteilt
- **Vorteil:** Konzeptuell ähnlich zur objektorientierten Programmierung
- **Nachteile:**
 - im allgemeinen geringere Performance als relationale Datenbanken
 - geringere Verbreitung

- gesellschaftliche Relevanz von Daten **nimmt stetig zu**
- Enorme **Datenmengen** speicherbar
- Herausforderung: sinnvolle **Strukturierung** und Verarbeitung der Daten

- Datenbanken ermöglichen **an die Realität anpassbare Struktur der Datenablage**

- **unterschiedliche** Datenbanktypen:
 - hierarchische Datenbank
 - Netzwerk Datenbank
 - **relationale Datenbank**
 - objektorientierte Datenbank

- Methoden zur **systematischen Erstellung** der Definition der an die Realität angepassten Struktur der Datenablage
 - Relationales Modell
 - Normalisierung
- **SQL** als (Programmier-)Sprache zur Interaktion mit einem Datenbank-Management-System
 - Abfrage und Manipulation von Daten
 - Erstellung und Manipulation der Datenablagestruktur
- ausgewählte Konzepte der Datenbanksystem **interne Arbeitsweise**
 - Indizes
 - Views
 - Transaktionssteuerung

1 Einführung

- Motivation
- Aufbau von Datenbanken
- Typen strukturorientierter DBS
- Organisatorisches

- Übung erfolgt an Rechnern im PC Pool → bitte sicherstellen, dass die persönlichen Zugangsdaten zu den Rechnern bekannt sind
- Vorbereitung der jeweiligen Termine notwendig, sonst keine Durchführung der Versuche sinnvoll
- Installationsanleitung der nötigen Software für das Üben zu Hause in Moodle verfügbar

- Moodle Kurs „Anwendung von Datenbanksystemen“
- Skript wird zur Verfügung gestellt

- 1 Einführung
- 2 Entity Relationship Modell
- 3 technisches Modell
- 4 Qualitätskriterien für relationales Modell
- 5 SQL - Übersicht und Data Definition Language
- 6 SQL - Data Definition Language Teil 2
- 7 SQL - Data Manipulation Language

Wiederholung

2 Entity Relationship Modell

- Modellierung
- Grundelemente
- erweiterte Elemente

Modellierung = **systematische Abbildung** realer Objekte, Vorgänge, Zusammenhänge, etc. zum Zweck der **Strukturierung der Datenhaltung**.

- Fokus auf **relevante** Informationen
- Vereinfachung
- Abstraktion

Modellebenen im Datenbankentwurfsprozess

- **konzeptionelles / logisches Modell**
 - **Entity Relationship Modell**
 - Diskussionsgrundlage für Kundengespräche
 - ggf. mehrere Modelle je nach Nutzergruppe → Nutzer-spezifische Sichten
- **technisches Modell**
 - **relationales Datenmodell**
 - logisches Modell + Strukturen zur Abbildung von Verknüpfungen
 - Basis zur Umsetzung der Tabellen Struktur
- **DB internes Modell**
 - DBS spezifisch
 - meist unabhängig vom technischen Modell

Entity-Relationship Modell

- verwendet Chen Notation²
- einfache, stark formalisierte Syntax und Semantik
- **grundlegende** Modellierungselemente:
 - Entitäten (genauer Entitätstypen)
 - Attribute
 - Schlüssel
 - Beziehungen
 - Kardinalitäten
- **erweiterte** Modellierungselemente:
 - Rollen
 - Beziehungsattribute
 - schwache Entitäten
 - abgeleitete Attribute

²Peter Pin-Shan Chen, „*The Entity-Relationship Model–Toward a Unified View of Data*“, 1976

2 Entity Relationship Modell

- Modellierung
- **Grundelemente**
- erweiterte Elemente

Entitätstypen

- bilden **Klassen** von physikalischen oder nicht physikalischen Objekten der Realität ab
- Notation: Rechteck
- Beispiele: Buch, Person, Haus, Bestellung, Rechnung

Attribute

- bilden **Eigenschaften** von Entitätstypen ab
- Notation: Ellipse - mit zugehörigem Entitätstyp verbunden
- Beispiele bei der Entität „Buch“: Titel, Autor, ISBN, Seitenzahl, ...

abgeleitete Attribute

- bilden Eigenschaften von Entitätstypen ab, die aus anderen Attributen **ohne zusätzliche Information** abgeleitet / berechnet werden können
- z.B. berechnet als Summe, Durchschnitt, Maximum, Minimum, etc.
- Notation: gestrichelte Ellipse

Schlüssel (-attribute)

- Attribut(e), die zur **eindeutigen Identifikation** einer Entität verwendet werden
- d.h. mehrere Objekte eines Entitätstyps mit denselben Werten der Schlüsselattribute sind ausgeschlossen!
- Identifikation an Hand **mehrerer Attribut** möglich
- kann bereits vorhandenes oder „künstlich“ erzeugtes Attribut sein
- Notation: Attribut(e) wird / werden unterstrichen
- Beispiel bei der Entität „Buch“: ISBN

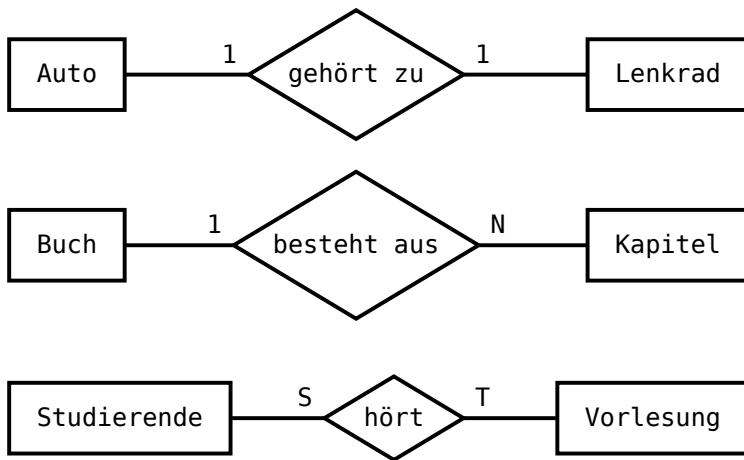
- bilden logische **Zusammenhänge** zwischen Entitätstypen ab
- Notation:
 - Raute mit Verbindung zu den in Beziehung stehenden Entitäten
 - innerhalb der Raute: Beschreibung der Beziehung



- bestimmt **wie viele** unterschiedliche Objekte eines Entitätstyps mit **wie vielen** Objekten eines anderen Entitätstyps **gleichzeitig** in Beziehung stehen können
- Chen Notation unterscheidet pro Beziehungsseite **nur zwei Optionen**:
 - 1: **ein Objekt** ist Teil der Beziehung
 - N (bzw. ein anderer eindeutiger Buchstabe): **mehrere Objekte** sind Teil der Beziehung
- pro Beziehung ergeben sich daraus **drei Möglichkeiten**:
 - $1 : 1 \rightarrow$ **ein** Objekt des Typs A steht mit **einem** Objekt des Typs B in Verbindung
 - $1 : N \rightarrow$ **ein** Objekt des Typs A steht mit **mehreren** Objekten des Typs B in Verbindung
 - $M : N \rightarrow$ **mehrere** Objekte des Typs A stehen mit **mehreren** Objekten des Typs B in Verbindung
- Notation: direkt an der Beziehung auf der Seite des zugehörigen Entitätstyps

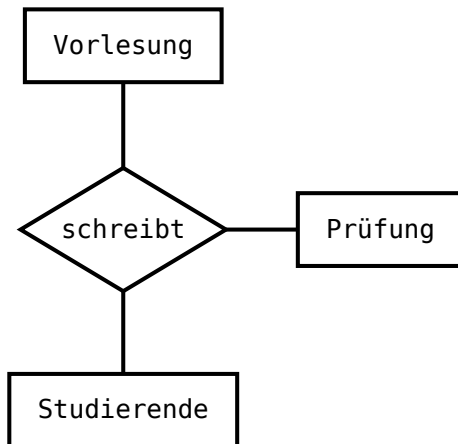
Kardinalitäten

Beispiele:



ternäre Beziehungen

- bei „klassischer“ Beziehung: **zwei** Entitäten stehen in Zusammenhang
- stehen mehr als zwei Entitäten in Zusammenhang → **ternäre** Beziehung



Bestimmung der Kardinalitäten bei ternären Beziehungen

Allgemeines Vorgehen:

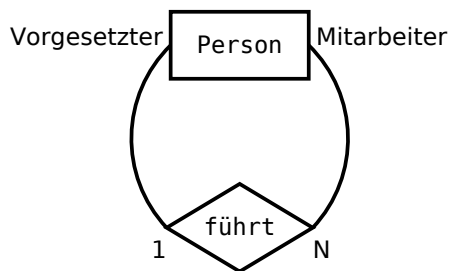
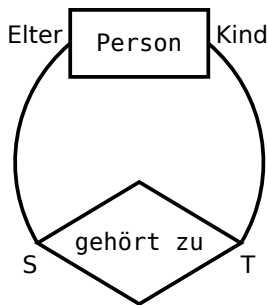
- zu klärende Frage: „Kann **eine** Entität des Typs A und **eine** Entität des Typs B mit **mehreren** Entitäten des Typs C in Beziehung stehen?“
- wenn „ja“ → Kardinalität des Typs C ist N
- wenn „nein“ → Kardinalität des Typs C ist 1
- Wiederholung mit vertauschten Rollen

Am Beispiel:

- Kann **ein** Studierender in **einer** Vorlesung **mehrere** Prüfungen schreiben? → ja (Zweitversuch) → Kardinalität bei Prüfungen M
- Kann **ein** Studierender **eine** Prüfung in **mehreren** Vorlesungen schreiben? → nein (Prüfungen unterscheiden sich zwischen den Fächern) → Kardinalität bei Vorlesung 1
- Kann **eine** Prüfung **einer** Vorlesung von **mehreren** Studierenden geschrieben werden? → ja → Kardinalität bei Studierenden N

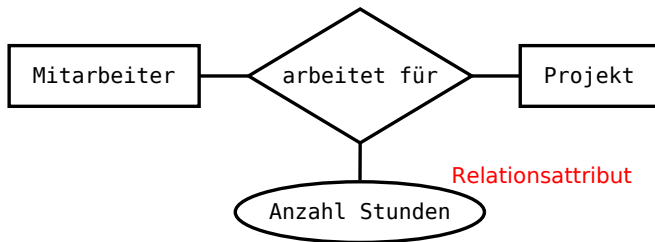
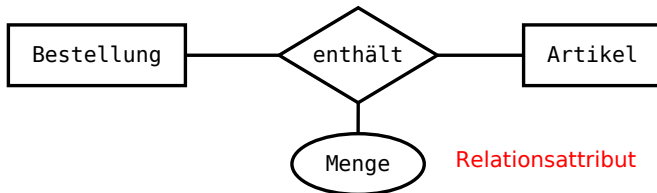
reflexive Beziehung und Rollen

- Objekte eines bestimmten Entitätstyps stehen mit Objekten **desselben** Entitätstyps in Beziehung
- die jeweiligen Objekte nehmen dabei unterschiedliche **Rollen** ein
- Notation: Beziehung deren Enden ein und denselben Entitätstyp verbinden. Rollen werden an der jeweiligen Verbindungslinie als Text notiert



Beziehungsattribute

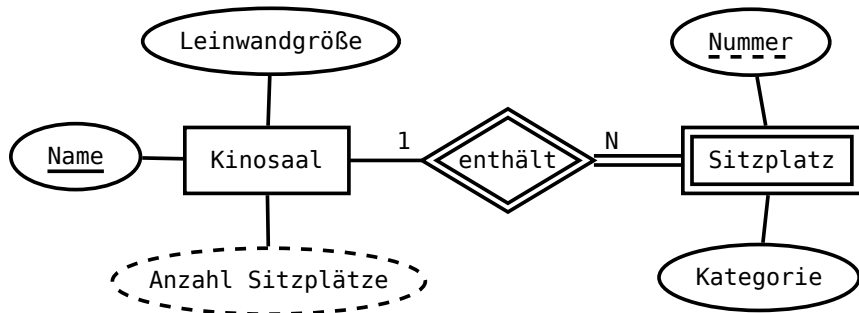
- Eigenschaften, die eine Beziehung näher beschreiben, können als Beziehungsattribute modelliert werden.



- alternative Bezeichnung: **abhängige** Entitätstypen
- Entitätstypen, deren Attribute **nicht ausreichen**, um ein Objekt eindeutig zu identifizieren
- Identifikation nur in Verbindung mit zugehörigem (starken) Entitätstyp möglich
- Kardinalität auf Seiten des starken Entitätstyps **muss 1 sein**.
Andernfalls keine Identifikation der schwachen Entität möglich
- Notation:
 - Entität: zweifacher Rahmen beim Rechteck, alternativ dicker Rechteckrahmen
 - Beziehung: zweifacher Rahmen bei Raute und doppelte Verbindungslinie von schwachem Entitätstyp zur Raute
 - Teilschlüssel Attribut(e): gestrichelt unterstrichen

schwache Entitätstypen

Beispiel:



- Sitzplatz Nummer ist bei Kinos mit mehreren Sälen nicht eindeutig
- Nummer Attribut ist deshalb nur Teilschlüssel

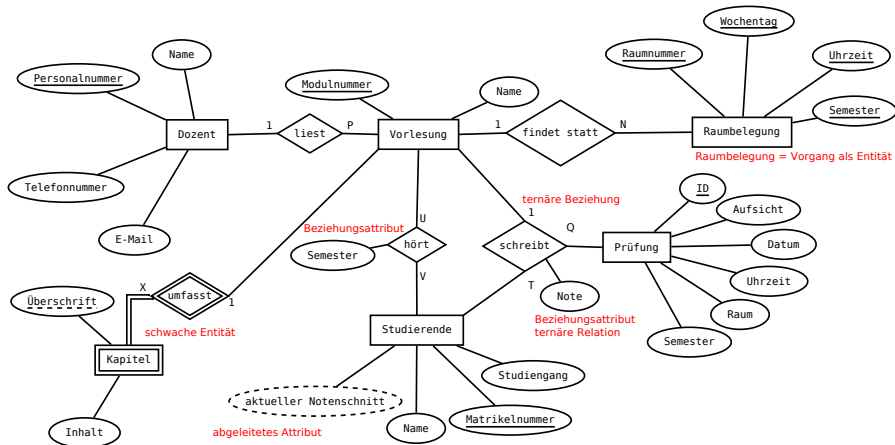
- Modellierung = **systematische Abbildung** der Realität zur **Strukturierung** der Datenhaltung
- Ebenen: **konzeptionell / logisch** → **technisch** → **DB intern**
- **Entity Relationship Modell** als typisches konzeptionelles Modell
- **Grundelemente:**
 - Entitätstyp
 - Attribut
 - Beziehung + Kardinalität
 - Schlüssel
- **erweiterte Elemente:**
 - ternäre Beziehung
 - reflexive Beziehung
 - Beziehungsattribut
 - schwache Entität

- 1 Einführung
- 2 Entity Relationship Modell
- 3 technisches Modell**
- 4 Qualitätskriterien für relationales Modell
- 5 SQL - Übersicht und Data Definition Language
- 6 SQL - Data Definition Language Teil 2
- 7 SQL - Data Manipulation Language

Wiederholung

Wiederholung - Teil 2

Beispiel eines Entity Relationship Modells:



Wie geht es weiter?

- konzeptionelles Modell (Entity Relationship Modell) ✓
- technisches Modell zur Verwendung in Datenbanksystem ?
- Leitfrage: wie können wir aus dem Entity Relationship Modell ein technisches Modell ableiten?

3 technisches Modell

- relationales Modell
- Überführung des ER Modells ins relationale Modell
- Vereinfachung des relationalen Modells
- Datentypen und optionale Attributwerte

- **Entity Relationship** Modelle verwenden **zwei** Strukturierungskonzepte:
 - Entitätstyp
 - Beziehungstyp
- **relationale Modelle** verwenden nur **ein** Strukturierungskonzept:
 - **Relation**

- tabellarische Darstellung des Zusammenhangs zwischen einzelnen Attributen

Relationsname

<u>Schlüssel Attribut 1</u>	Attribut 2	Attribut 3	...
A	B	C	...
F	G	H	...

- Spalten werden als **Attribute** bezeichnet
- Zeilen werden als **Tupel** bezeichnet

Achtung

Unterschiedliche Bedeutung des Begriffs **Relation**!

- bei Entity Relationship Modellen: Beziehung zwischen Entitäten
- bei relationalem Modell: Tabelle

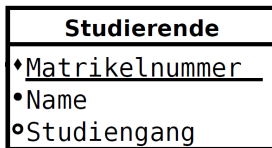
Darstellungsformen einer Relation

- tabellarische Darstellung

Studierende

<u>Matrikelnummer</u>	Name	Studiengang
4711	Meyer	WIN
0815	Müller	WIT
4242	Schulze	KIK

- Modell Darstellung



- formale Darstellung

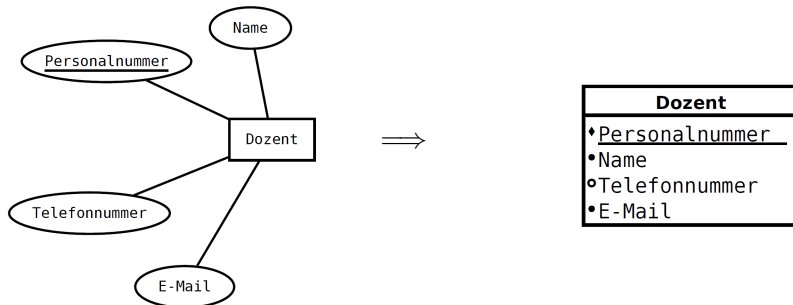
Studierende : { Matrikelnummer, Name, Studiengang }

3 technisches Modell

- relationales Modell
- Überführung des ER Modells ins relationale Modell
- Vereinfachung des relationalen Modells
- Datentypen und optionale Attributwerte

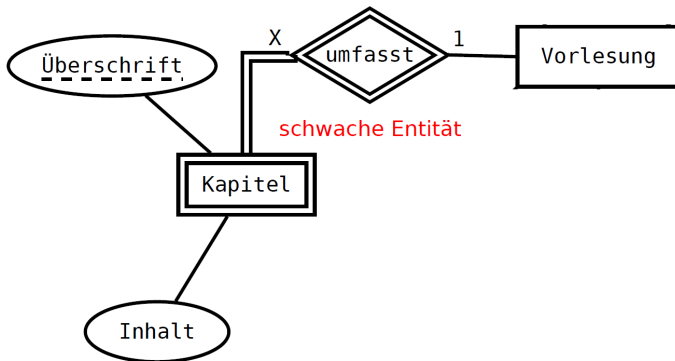
Entitätstypen

- jeder **Entitätstyp** im ER Modell wird zu einer Relation im relationalen Modell
- Name des Entitätstyps wird zum Namen der Relation
- Attribute des Entitätstyps werden zu Attributen der Relation
- Schlüsselattribute des Entitätstyps werden zu Schlüsselattributen der Relation
- Beziehungen zwischen Entitätstypen besprechen wir später

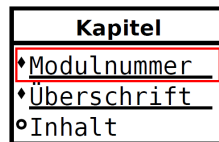


- jeder **schwache Entitätstyp** im ER Modell wird zu einer Relation im relationalen Modell
- Name des schwachen Entitätstyps wird zum Namen der Relation
- Attribute des schwachen Entitätstyps werden zu Attributen der Relation
- **Teilschlüsselattribute** des schwachen Entitätstyps werden zu **Schlüsselattributen** der Relation
- der Schlüssel der Relation wird um **alle Primärschlüssel Attribute** des **übergeordneten Entitätstyps** erweitert
- → Der Primärschlüssel der Relation eines schwachen Entitätstyps ist **immer** aus mehreren Attributen zusammengesetzt!

schwache Entitätstypen - Ein Beispiel

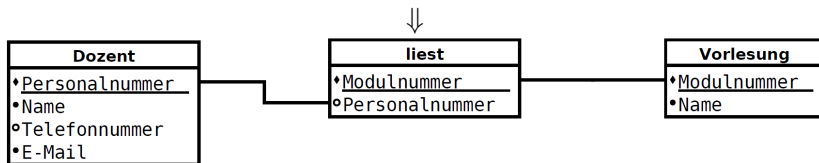
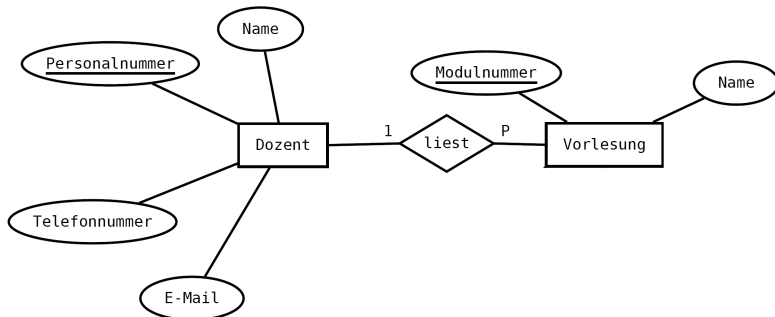


- „Modulnummer“ ist das Schlüsselattribut des Entitätstyps „Vorlesung“

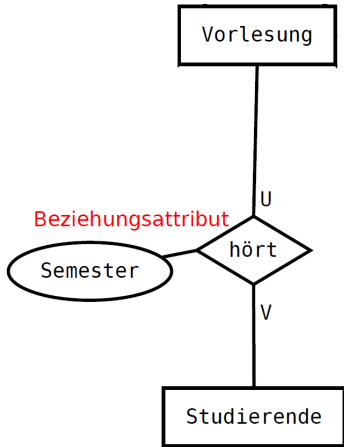


- jeder **Beziehungstyp** im ER Modell wird zu einer **Relation** im relationalen Modell
- **alle Schlüsselattribute aller** in Beziehung stehender Entitätstypen werden zu Attributen der Relation
- referenzieren die jeweiligen Tupel in den Relationen der Entitätstypen
→ **Fremdschlüssel**
- **alle Schlüsselattribute** der Entitätstypen mit Kardinalität > 1 werden zum **Schlüssel der Relation**
- Attribute des Beziehungstyps werden zu (Nicht-Schlüssel) Attributen der Relation

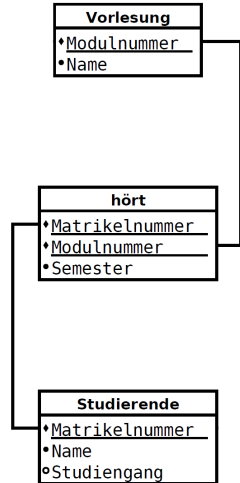
Beziehungstypen - Ein Beispiel



Beziehungstypen - Ein weiteres Beispiel

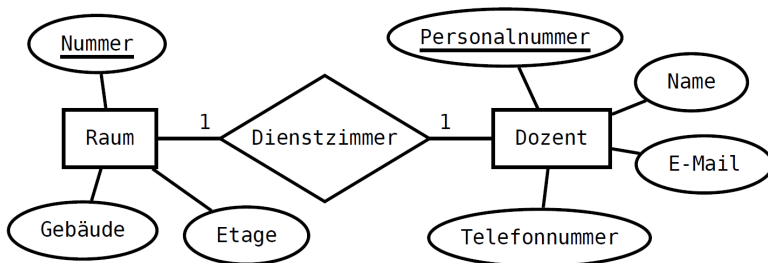


⇒



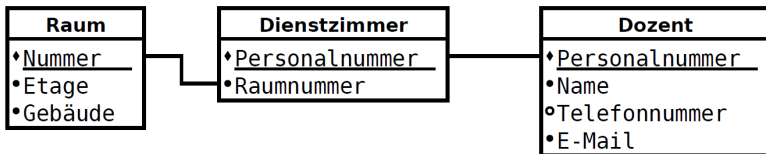
Beziehungstypen - Sonderfall 1:1 Kardinalität

- bei Kardinalität 1:1 hätte nach den bisherigen Regeln die Relation des Beziehungstyps keinen Schlüssel
- Schlüsselattribut der Relation kann aus den von den Entitätstypen übernommenen Schlüsselattributen **gewählt werden**
- Beispiel

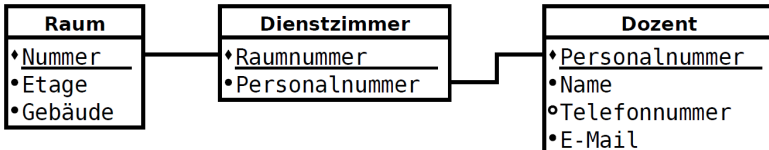


Beziehungstypen - Sonderfall 1:1 Kardinalität

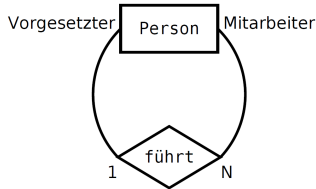
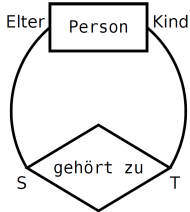
- die Wahl des Schlüssels bei einer 1:1 Kardinalität ist **grundsätzlich frei**
- Berücksichtigung **Anwendungsdomänen spezifischer Vorgaben** sinnvoll
- meist wird der Schlüssel der „volatileren“ Entität gewählt
- „Personalnummer“ als Schlüssel:



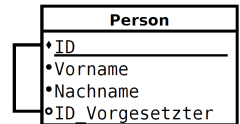
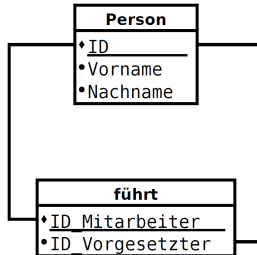
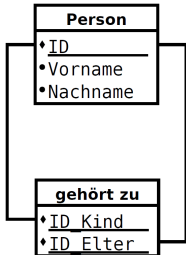
- „Raumnummer“ als Schlüssel:



reflexiver Beziehungstyp



- **Rollennamen** zu Schlüsselnamen hinzufügen

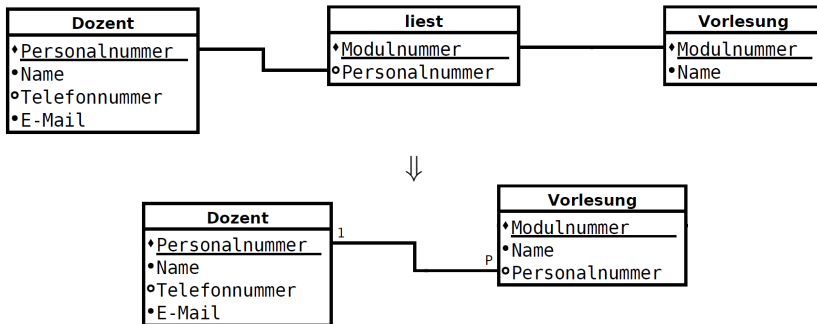


3 technisches Modell

- relationales Modell
- Überführung des ER Modells ins relationale Modell
- Vereinfachung des relationalen Modells
- Datentypen und optionale Attributwerte

Vereinfachung des relationalen Modells

- Bei Beziehungen mit Kardinalität **1:1**, **1:N** und **N:1** lassen sich die Beziehungstyp Relationen **meist zusammenfassen**
- Beispiel: Relation „liest“ kann in „Vorlesung“ integriert werden

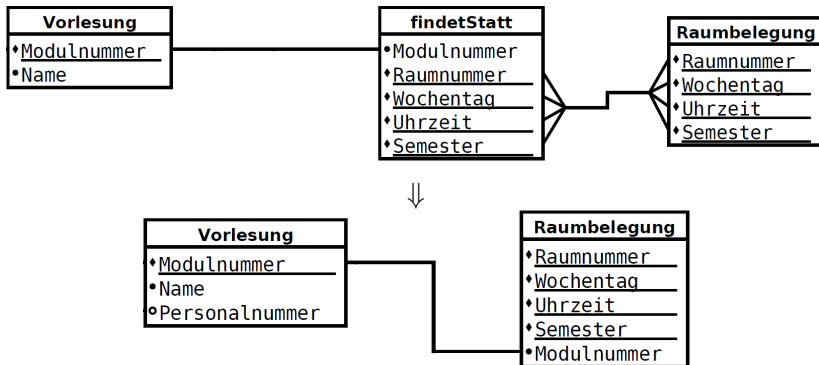


Wichtig

Es dürfen nur Relationen **mit identischem Schlüssel** zusammengefasst werden, die zueinander in Beziehung stehen

Schlüssel aus mehreren Attributen

- Das Zusammenfassen von Relationen ist auch bei Schlüsseln aus **mehreren Attributen** zulässig.
- Beispiel: „findetStatt“ kann in „Raumbelegung“ integriert werden

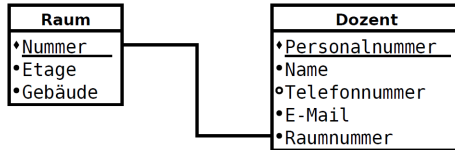


Wichtig

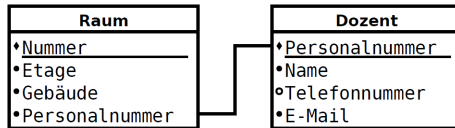
alle Schlüsselattribute müssen übereinstimmen

Vereinfachung des relationalen Modells bei 1:1 Kardinalität

- Abhängig von der **Wahl des Schlüssel Attributs** lässt sich die Beziehungstyp Relation mit der einen oder anderen Entitätstyp Relation zusammenfassen
- „Personalnummer“ als Schlüssel erlaubt Integration in „Dozent“ Relation:



- „Raumnummer“ als Schlüssel erlaubt Integration in „Raum“ Relation:



- welche Variante ist aus Anwendungssicht sinnvoller?

3 technisches Modell

- relationales Modell
- Überführung des ER Modells ins relationale Modell
- Vereinfachung des relationalen Modells
- Datentypen und optionale Attributwerte

- werden Daten im Rechner gespeichert, haben diese Daten immer einen **Datentyp**
- Die wichtigsten Datentypen im **Überblick**:
 - **INT**: ganze Zahlen
 - **DECIMAL**(n,m): Festkommazahlen mit n Stellen, davon m Nachkommastellen
 - **FLOAT**: Fließkommazahlen
 - **CHAR**(n): Zeichenkette der (exakten) Länge n
 - **VARCHAR**(n): Zeichenkette variabler aber maximaler Länge n
 - **DATE**: Datum (Jahr, Monat, Tag)
 - **DATETIME**: Zeitpunkt im Format YYYY-MM-DD hh:mm:ss[.fraction] im Bereich 1000-01-01 00:00:00.000000 - 9999-12-31 23:59:59.999999
 - **TIMESTAMP**: Zeitpunkt im Format YYYY-MM-DD hh:mm:ss[.fraction] im Bereich 1970-01-01 00:00:01.000000 - 2038-01-19 08:44:07.999999 mit Unterstützung von **Zeitzone**n inkl. automatischer Konvertierung
 - **BOOLEAN**: Wahrheitswert
 - **BLOB**: „**B**inary **L**arge **O**bjekt“

→ verfügbare Datentypen hängen vom eingesetzten DBMS ab

Darstellung im relationalen Modell

- werden **pro Attribut** angegeben
 - Darstellung rechts des Attributnames
 - für Schlüssel Attribute eignen sich insb. **ganzzahlige Datentypen** oder **Zeichenketten**
- gilt sowohl für **Primärschlüssel** als auch für **Fremdschlüssel**
- Datentyp des Fremdschlüssel Attributs muss zu Datentyp des Primärschlüssel Attributs passen
- Beispiel:

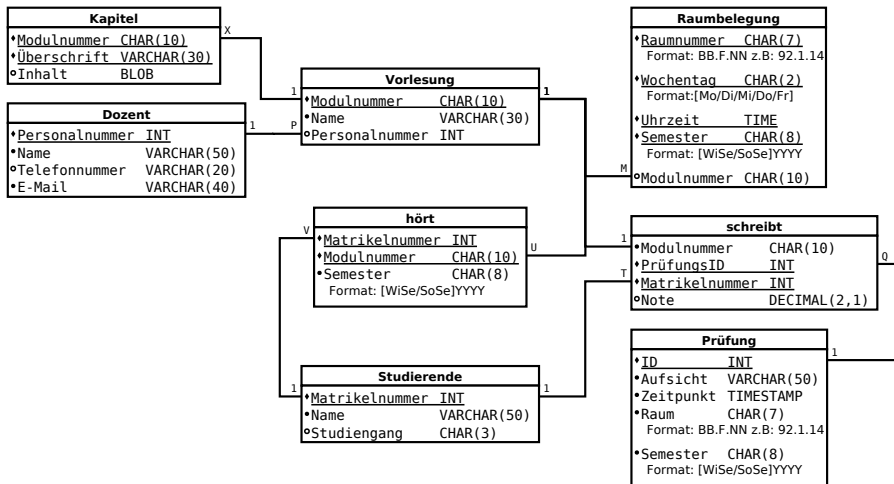
Dozent	
♦ <u>Personalnummer</u>	INT
• Name	VARCHAR(50)
◦ Telefonnummer	VARCHAR(20)
• E-Mail	VARCHAR(40)

Vorlesung	
♦ <u>Modulnummer</u>	CHAR(10)
• Name	VARCHAR(30)
◦ Personalnummer	INT

- Attribute können als **nullable** markiert werden
- die Angabe eines Wertes für dieses Attribut ist damit **optional**
- Umkehrschluss: alle Attribute die **nicht nullable** sind, erfordern **zwingend** die Angabe eines konkreten Wertes bei der Erstellung eines Tupels
- nullable Werte werden üblicherweise mit einem nicht ausgefüllten Bulletpoint dargestellt (siehe Attribut „Telefonnummer“)

Dozent	
♦ <u>Personalnummer</u>	INT
• Name	VARCHAR(50)
◦ Telefonnummer	VARCHAR(20)
• E-Mail	VARCHAR(40)

relationales Beispiel Modell



- konzeptionelles **Entity Relationship** Modell muss in technisches **relationales Modell** überführt werden
- **Relation** als einziges Struktur Element im relationalen Modell
- Entitätstyp → Relation
- Beziehungstyp → Relation(!)
- je nach Kardinalität **Zusammenfassung** von Relationen möglich
- erfordert **identische Schlüssel** in beiden Relationen
- **Datentyp** für jedes Attribut im technischen Modell
- **nullable** Attribute erfordern keinen Wert bei der Erstellung / Manipulation eines Tupels

- 1 Einführung
- 2 Entity Relationship Modell
- 3 technisches Modell
- 4 Qualitätskriterien für relationales Modell**
- 5 SQL - Übersicht und Data Definition Language
- 6 SQL - Data Definition Language Teil 2
- 7 SQL - Data Manipulation Language

Wiederholung

4 Qualitätskriterien für relationales Modell

- Anomalien
- funktionale Abhängigkeit
- Normalformen

- als **Anomalie** wird ein unerwünschtes Verhalten bei der Nutzung einer Datenbank bezeichnet
- Abhängig vom **Aufbau des Datenbankmodells** kann es zu sog. **Anomalien** kommen

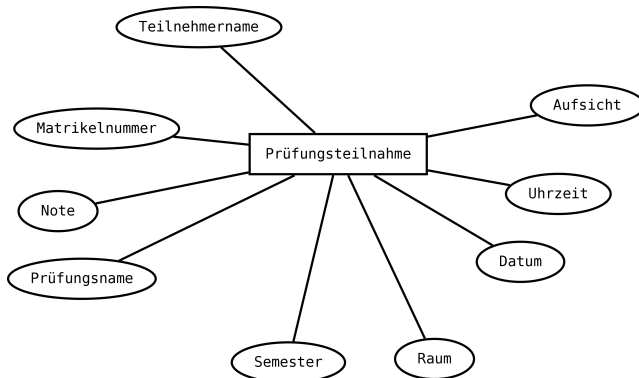
Anomalietypen:

- **Updateanomalie**
- **Einfügeanomalie**
- **Löschanomalie**

→ Ziel ist es durch **geschickte Modellerstellung** diese Anomalien **von vornherein auszuschließen**

Beispielmodell mit Anomalien

- Anomalien treten meist auf, wenn mehrere in der realen Welt **unabhängige Elemente** als eine **einzige Entität** mit allen Attributen der unabhängigen Elemente modelliert werden:



Updateanomalie

- Änderungen eines **einzigen Attributwertes** müssen an **mehreren Stellen** in der Datenbank vorgenommen werden
- resultierende Probleme:
 - Gefahr, dass min. eine Stelle übersehen wird → Datenbasis wird **inkonsistent**
 - **erhöhter Speicherbedarf** durch mehrfache Speicherung desselben Wertes
 - Updateoperation benötigen **mehr Zeit**
- Beispiel: Prüfungsinformationen werden für jeden Teilnehmer **redundant gespeichert**

Teiln.Name	Mat.Nr	Note	Prf.name	Sem.	Raum	Datum	Zeit	Aufsicht
Meyer	4711	1,7	Mathe	WS 22	92.0.16	23.01.23	11:00	Huber
Müller	0815	2,0	Mathe	WS 22	92.0.16	23.01.23	11:00	Huber
Schulze	4242	1,3	Mathe	WS 22	92.0.16	23.01.23	11:00	Huber

- Das Einfügen eines **neuen Datensatzes** ist **nicht möglich**
- mögliche Ursachen:
 - Wert des **Primärschlüssels** existiert bereits
 - es sind **noch nicht alle Werte** der notwendigen Attribute **bekannt**
- Beispiel: neue Prüfung, bei der sich noch niemand angemeldet hat

Teiln.Name	Mat.Nr	Note	Prf.name	Sem.	Raum	Datum	Zeit	Aufsicht
Meyer	4711	1,7	Mathe	WS 22	92.0.16	23.01.23	11:00	Huber
Müller	0815	2,0	Mathe	WS 22	92.0.16	23.01.23	11:00	Huber
Schulze	4242	1,3	Mathe	WS 22	92.0.16	23.01.23	11:00	Huber
???	???	???	Physik	WS 22	51.1.2	27.01.23	08:00	Dietrich

- Beim Löschen eines Datensatzes werden **mehr Informationen** gelöscht **als notwendig**
- Beispiel: werden alle Studierenden, die eine bestimmte Prüfung abgelegt haben, aus der Datenbank gelöscht, da sie ihr Studium beendet haben, gehen alle Prüfungs-bezogenen Informationen verloren

Teiln.Name	Mat.Nr	Note	Prf.name	Sem.	Raum	Datum	Zeit	Aufsicht
MeyerMeyer	47114711	1,71,7	MatheMathe	WS 22WS-22	92.0.1692-0-16	23.01.2323-01-22	11:0011:00	HuberHuber
MüllerMüller	08150815	2,02,0	MatheMathe	WS 22WS-22	92.0.1692-0-16	23.01.2323-01-22	11:0011:00	HuberHuber
SchulzeSchulze	42424242	1,31,3	MatheMathe	WS 22WS-22	92.0.1692-0-16	23.01.2323-01-22	11:0011:00	HuberHuber
Leis	12345	2,7	Physik	WS 22	51.1.2	27.01.23	08:00	Dietrich

4 Qualitätskriterien für relationales Modell

- Anomalien
- funktionale Abhängigkeit
- Normalformen

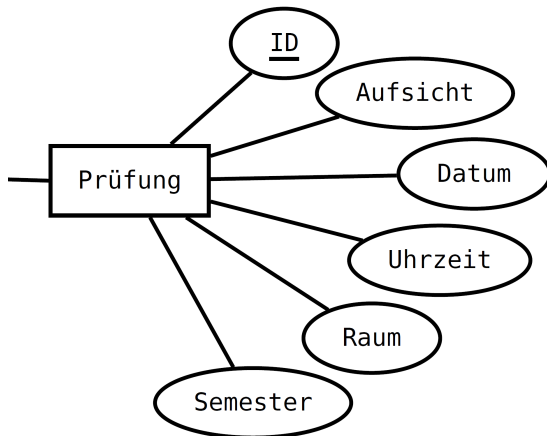
- beschreibt Zusammenhänge zwischen Attribut**werten**

Definition

bei einer **funktionalen Abhängigkeit** zwischen den zwei Attributmengen α und β sind bei bekannten Werten für α die Werte der Attribute in β eindeutig festgelegt

- anderes ausgedrückt: Wenn mehrere Entitäten existieren, deren Attribute α identische Werte aufweisen, **müssen** auch die Attribute in β identische Werte aufweisen
- **formale Notation:** $\alpha \rightarrow \beta$
- Der Pfeil kann als „bestimmt eindeutig“ gelesen werden
- α wird auch als **Determinante** von β bezeichnet

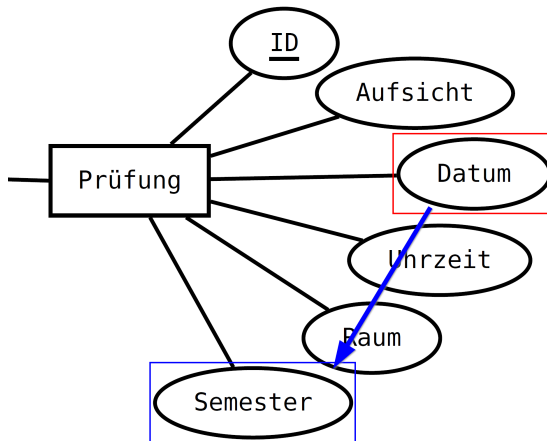
- siehe Entitätstyp Prüfung:



- welche funktionalen Abhängigkeiten existieren?

funktionale Abhängigkeit am Beispiel

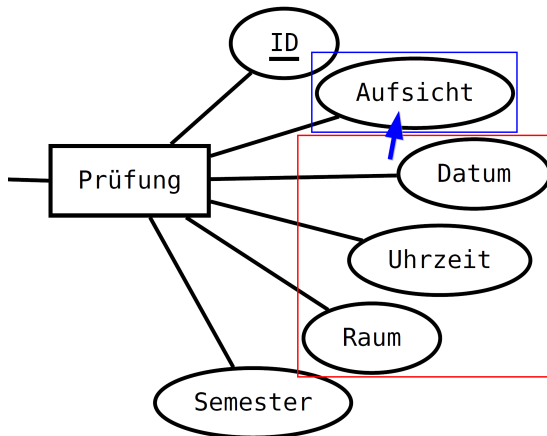
- $\{\text{Datum}\} \rightarrow \{\text{Semester}\}$



- **Datum** bestimmt eindeutig das **Semester** in welchem die Prüfung stattfindet

funktionale Abhängigkeit am Beispiel

- $\{\text{Datum, Uhrzeit, Raum}\} \rightarrow \{\text{Aufsicht}\}$



- Die Kombination aus **Datum, Uhrzeit und Raum** bestimmt eindeutig die **Aufsicht**

Zusammenhang zwischen funktionaler Abhängigkeit und Schlüsseln

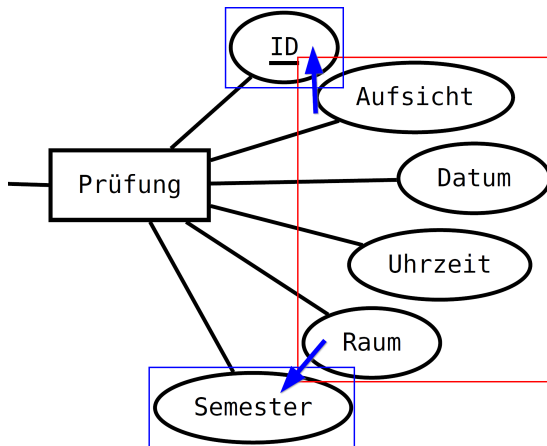
Definition

Ein **Superschlüssel** ist eine **Menge von Attributen** eines Entitätstyps, die eindeutig **alle** anderen verbleibenden Attribute des Entitätstyps bestimmen

- d.h. alle **verbleibenden** Attribute eines Entitätstyps sind **funktional abhängig** vom Superschlüssel
- trivial: die Menge aller Attribute einer Entität ist automatisch ein Superschlüssel

Superschlüssel am Beispiel

- Beispiel eines Superschlüssels:

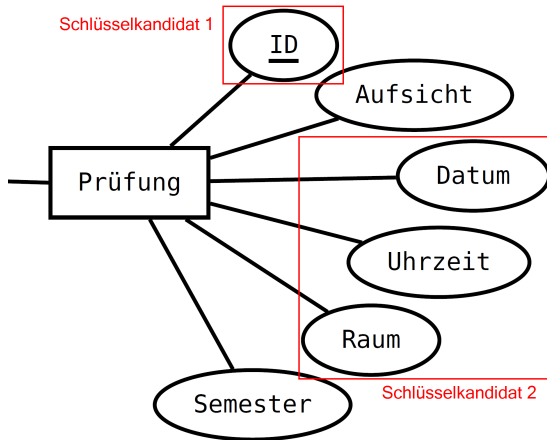


- formal: $\{\text{Datum, Uhrzeit, Raum, Aufsicht}\} \rightarrow \{\text{Semester, ID}\}$

- im vorigen Beispiel ist die Determinante $\{\text{Datum, Uhrzeit, Raum, Aufsicht}\}$ zur eindeutigen Bestimmung der verbleibenden Attribute **größer als nötig**.
- Das Attribut „Aufsicht“ wird in der Determinante nicht benötigt, um alle anderen Attribute eindeutig zu bestimmen!
- **volle funktionale Abhängigkeit** gegeben, wenn Determinante des Superschlüssels **möglichst klein**
- Determinanten, von denen alle anderen Attribute des Entitätstyps **voll** funktional abhängig sind, werden als **Schlüsselkandidaten** oder **Kandidatenschlüssel** bezeichnet

Schlüsselkandidaten am Beispiel

- Schlüsselkandidat 1: {ID}
- Schlüsselkandidat 2: {Datum, Uhrzeit, Raum}



→ einer der Schlüsselkandidaten wird als **Primärschlüssel** ausgewählt.

4 Qualitätskriterien für relationales Modell

- Anomalien
- funktionale Abhängigkeit
- Normalformen

- **Normalisierung** überführt das relationale Modell in **Normalformen** durch **Eliminierung** von Anomalien / Redundanzen
 - existierende Normalformen:
 - 1. Normalform
 - 2. Normalform
 - 3. Normalform
 - Boyce Codd Normalform
 - 4. Normalform³
 - 5. Normalform³
 - Normalformen bauen aufeinander auf
- 2. NF umfasst 1. NF, 3. NF umfasst 2. NF, etc.

³spielt in der Praxis keine Rolle

1. Normalform

Definition

alle Attribute haben **atomare Wertebereiche**

- d.h. zusammengesetzte oder Mengen-wertige Attribute sind verboten
- Vorteil: alle relevanten Informationen sind **direkt ansprechbar**
- Beispiele

nicht in 1. Normalform:

Prf.name	Zeitpunkt
Mathe	23.01.23 11:00
Englisch	23.01.23 14:00
Physik	27.01.23 08:00

nicht in 1. Normalform:

Dozent	Vorlesungen
Mayer	Mathe, Physik
Müller	Englisch

1. Normalform:

Prf.name	Datum	Zeit
Mathe	23.01.23	11:00
Englisch	23.01.23	14:00
Physik	27.01.23	08:00

1. Normalform:

Dozent	Vorlesung
Mayer	Mathe
Mayer	Physik
Müller	Englisch

2. Normalform

Definition

- Relation befindet sich in 1. Normalform **und**
 - Jedes **Nichtschlüssel Attribut** ist **voll funktional abhängig** von jedem **Schlüsselkandidaten** der Relation
- Übersetzt: bei jedem Schlüsselkandidaten sind alle seine Attribute notwendig, um die restlichen Attribute der Relation eindeutig zu bestimmen
- es darf keine funktionale Abhängigkeit von Attributen von Teilen eines Schlüsselkandidaten geben!
- gut zu wissen: bestehen alle Schlüsselkandidaten einer Relation in der 1. NF aus nur einem Attribut ist die 2. NF automatisch erreicht

2. Normalform - graphische Veranschaulichung

3. Normalform

Definition

- Relation befindet sich in 2. Normalform **und**
- kein **Nichtschlüssel Attribut** hängt **transitiv** von einem Kandidatenschlüssel ab
- anders formuliert: kein **Nichtschlüssel Attribut** darf andere Attribute der Relation eindeutig bestimmen

3. Normalform - graphische Veranschaulichung

Definition

- Relation befindet sich in 3. Normalform **und**
- jede Determinante ist ein Kandidatenschlüssel der Relation
- anders formuliert: es gibt **ausschließlich** funktionale Abhängigkeiten von allen Kandidatenschlüsseln
- gut zu wissen: die BCNF muss nur angewendet werden, wenn in einer Relation **mehrere** Kandidatenschlüssel existieren, von denen mindestens einer **mehrwertig** ist

BCNF - graphische Veranschaulichung

- **Anomalie** = unerwünschtes Verhalten bei der Nutzung einer Datenbank
 - Einfügeanomalie
 - Updateanomalie
 - Löschanomalie
- **funktionale Abhängigkeiten** definieren Zusammenhänge zwischen Attributen einer Relation
- ein **Superschlüssel** bestimmt eindeutig **alle anderen** Attribute einer Relation
- ein **Schlüsselkandidat** ist eine Attributmenge von der alle anderen Attribute **voll** funktional abhängig sind
- **Normalisierung** eliminiert Anomalien und Redundanzen im Datenmodell
- übliche Normalformen: **1., 2., 3. Normalform, Boyce Codd Normalform**

- 1 Einführung
- 2 Entity Relationship Modell
- 3 technisches Modell
- 4 Qualitätskriterien für relationales Modell
- 5 SQL - Übersicht und Data Definition Language**
- 6 SQL - Data Definition Language Teil 2
- 7 SQL - Data Manipulation Language

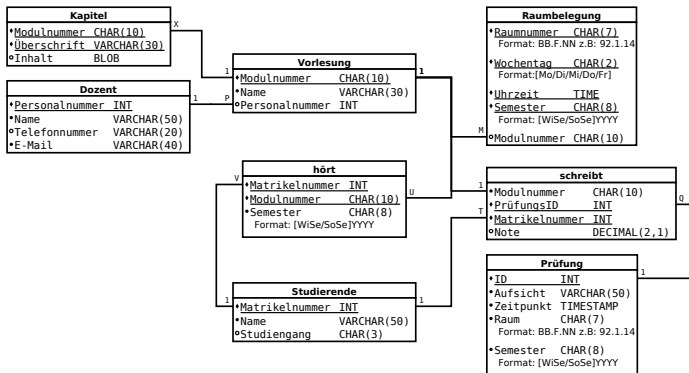
Wiederholung

5 SQL - Übersicht und Data Definition Language

- Übersicht
- Schemaerstellung mit Data Definition Language
- Tabellenerstellung mittels Data Definition Language

Implementierung des technischen Modells

- (normalisiertes) technisches Modell = formale Definition der **Anwendungsbereichs-spezifischen** Datenstruktur



- nächster Schritt: Implementierung des technischen Modells unter Verwendung eines **universell einsetzbaren** DBMS

Structured Query Language

- **SQL** = **S**tructured **Q**uery **L**anguage
- vormalig „**SEQUEL**“: Structured **E**nglish Query Language
- Erste **Standardisierung** in 1986
- **Deklarative** und **Imperative** Sprachanteile
- SQL Sprachbestandteile werden üblicherweise in GROSSBUCHSTABEN geschrieben
- jedes SQL Statement wird mit einem „;“ abgeschlossen
- mehrere Statements können in einem **SQL Skript** zusammengefasst und der Reihe nach ausgeführt werden
- Reference Manual für MySQL und MariaDB:
<https://dev.mysql.com/doc/refman/8.0/en/>

Kategorien von SQL Befehlen

SQL umfasst mehrere **Kategorien** von Statements:

- **Data Definition Language (DDL)**: Definition der **Struktur** der Datenbasis
- **Data Manipulation Language (DML)**: Manipulation der **Inhalte** der Datenbasis
- **Data Query Language (DQL)**: Abfrage von **Inhalten** der Datenbasis
- **Data Transaction Language (DTL)**: Steuerung der atomaren Transaktionsausführung
- **Data Control Language (DCL)**: **Administration** des Datenbanksystems

- 5 SQL - Übersicht und Data Definition Language
 - Übersicht
 - Schemaerstellung mit Data Definition Language
 - Tabellenerstellung mittels Data Definition Language

- Dient zur Verwaltung der Struktur der Datenbasis
- Strukturelemente (Auswahl):
 - **DATABASE** = Schema innerhalb der Datenbasis
 - **TABLE** = **Relation** aus relationalem Modell
 - **VIEW** = Anwendungsspezifische **Sicht** auf Datenbasis
 - **INDEX** = „Inhaltsverzeichnis“ für schnelle Suche
 - **FUNCTION** und **PROCEDURE** = wiederverwendbare SQL Fragmente
 - **TRIGGER** = Automatische Ausführung von SQL Befehlen bei Manipulationen der Inhalte der Datenbasis
- verfügbare Operationen
 - Anlegen mittels **CREATE**
 - Ändern mittels **ALTER**
 - Ausgabe mittels **DESCRIBE**
 - Löschen mittels **DROP**

- Syntax (vereinfacht):

```
CREATE DATABASE [IF NOT EXISTS] <schemaname>;
```

- statt **DATABASE** kann das Schlüsselwort **SCHEMA** verwendet werden
- wird **IF NOT EXISTS** nicht angegeben, wird ein Fehler erzeugt, falls ein Schema dieses Namens existiert
- Statement **erweiterbar** um Angaben zu⁴:
 - Zeichensatz
 - Kollation
 - Verschlüsselung der Datenbasis

⁴Details siehe <https://dev.mysql.com/doc/refman/8.0/en/>

- Syntax:

DROP DATABASE [IF EXISTS] <schemaname>;

- entfernt das Schema inkl. all seiner Bestandteile → enthaltene Elemente müssen nicht separat gelöscht werden

Achtung

Es erfolgt keine Rückfrage, ob das Schema gelöscht werden soll!

Achtung 2

DROP Statements können nicht rückgängig gemacht werden!

- die Datenbasis kann mehrere Schemata enthalten
- ggf. Elemente (Tabellen, Views etc.) mit **identischem Namen** in **unterschiedlichen** Schemata
- Verwendung eines Elements erfordert Angabe des vollqualifizierten Namen: `<schemaname>.<elementname>`
- werden häufig Elemente aus dem gleichen Schema verwendet, kann dieses Schema per USE Statement eingebunden werden
- Syntax:
`USE <schemaname>;`

- bei der Entwicklung eines neuen Schemas ist folgendes Pattern hilfreich

```
DROP DATABASE IF EXISTS <schemaname>;
```

```
CREATE DATABASE <schemaname>;
```

```
USE <schemaname>;
```

- 5 SQL - Übersicht und Data Definition Language
 - Übersicht
 - Schemaerstellung mit Data Definition Language
 - Tabellenerstellung mittels Data Definition Language

- Syntax (vereinfacht):

```
CREATE TABLE [IF NOT EXISTS] <tablename> (  
    create_definition,  
    [create_definition, ...]  
);
```

- mindestens eine create_definition
- gültige create_definitions:
 - **Spaltendefinition**
 - Indexdefinition
 - **Primärschlüsseldefinition**
 - **Fremdschlüsseldefinition**
 - **Check Constraint Definition**

Spaltendefinition

- Syntax (vereinfacht):

```
<column_name> <data_type> [NOT NULL | NULL]
    [DEFAULT <default_value>] [AUTO_INCREMENT]
    [UNIQUE] [PRIMARY KEY] [COMMENT '...']
```

- Datentypen siehe nächste Folie
- **NOT NULL** → Wert für das Attribut **muss** angegeben werden
- **NULL** → Wert für das Attribut ist **optional**
- **DEFAULT** → ermöglicht Vorgabe eines Standardwertes, falls beim Erstellen eines neuen Eintrags kein Wert für das Attribut angegeben wird
- **AUTO_INCREMENT** → zählt Primärschlüssel Spalte automatisch hoch
- **UNIQUE** → Attributwerte müssen eindeutig sein, NULL Werte sind erlaubt (außer sie sind mittels **NOT NULL** explizit ausgeschlossen)
- **PRIMARY KEY** → Attribut dient als Primärschlüssel

→ nur möglich bei Primärschlüsseln bestehend aus **einem** Attribut

- **INT**: ganze Zahlen
- **DECIMAL**(n,m): Festkommazahlen mit n Stellen, davon m Nachkommastellen
- **FLOAT**: Fließkommazahlen
- **CHAR**(n): Zeichenkette der (exakten) Länge n
- **VARCHAR**(n): Zeichenkette variabler aber maximaler Länge n
- **DATE**: Datum (Jahr, Monat, Tag)
- **DATETIME**: Zeitpunkt im Format YYYY-MM-DD hh:mm:ss[.fraction] im Bereich 1000-01-01 00:00:00.000000 - 9999-12-31 23:59:59.999999
- **TIMESTAMP**: Zeitpunkt im Format YYYY-MM-DD hh:mm:ss[.fraction] im Bereich 1970-01-01 00:00:01.000000 - 2038-01-19 08:44:07.999999 mit Unterstützung von **Zeitzone**n inkl. automatischer Konvertierung
- **BOOLEAN**: Wahrheitswert
- **BLOB**: „**B**inary **L**arge **O**bjekt“

Primärschlüssel vs. Fremdschlüssel

- **Primärschlüssel** dient zur **eindeutigen Identifikation** aller Datensätze einer Relation
 - bezieht sich auf die **eigene** Relation
 - Wert des Primärschlüssel für jeden Datensatz **eindeutig**
- **Fremdschlüssel** dient zur **Verknüpfung** von Datensätzen einer Relation mit Datensätzen einer anderen Relation
 - bezieht sich auf **referenzierte** Relation
 - Wert des Fremdschlüsselattributs muss **nicht eindeutig** sein
- Hinweis: ein Attribut kann **sowohl** Primärschlüssel der eigenen Relation **als auch** ein Fremdschlüssel der sich auf eine andere Relation bezieht sein

Primärschlüsseldefinition

2 Optionen:

- als Teil der Spaltendefinition, z.B:

```
CREATE TABLE <table_name> (  
    <column_name1> INT PRIMARY KEY,  
    <column_name2> VARCHAR(50),  
    ...  
);
```

→ nur bei Primärschlüsseln aus einem Attribut möglich

- als eigene create_definition, z.B:

```
CREATE TABLE <table_name> (  
    <column_name1> INT,  
    <column_name2> VARCHAR(50),  
    ...  
    PRIMARY KEY(<column_name1>)  
);
```

→ bei **mehrwertigen Primärschlüsseln** notwendig

Fremdschlüsseldefinition

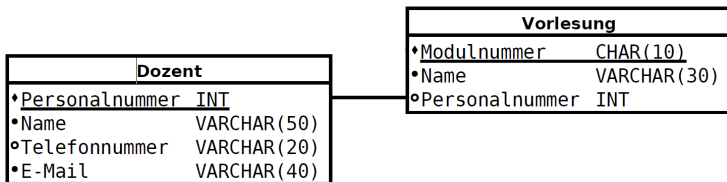
- Syntax:

```
CREATE TABLE <local_table> (  
    <local_column_name> <data_type>  
    ...  
    FOREIGN KEY (<local_column_name>  
        REFERENCES <remote_table>(<remote_column_name>)  
        [ON UPDATE <reference_option>]  
        [ON DELETE <reference_option>]  
);
```

- Datentyp von `local_column_name` muss dem Datentyp der referenzierten `remote_column_name` in `remote_table` entsprechen!
- `reference_options`
 - **RESTRICT** (= Default) verhindert das Ändern bzw. Löschen von Tupeln in der referenzierten Tabelle, solange auf diese referenziert wird
 - **CASCADE** aktualisiert bzw. löscht die Tupel in der lokalen Tabelle
 - **SET NULL** setzt das Fremdschlüsselattribut auf NULL, wenn der Wert in referenzierter Tabelle geändert bzw. gelöscht wird

- Fremdschlüssel Beziehungen zwischen Relationen stellen die **referenzielle Integrität** der Datenbasis sicher
- DBMS prüft beim Einfügen eines neuen Tupels, ob **alle** Fremdschlüsselattribute des neuen Tupels auf **existierende Datensätze** in den referenzierten Tabellen „zeigen“
- ist dies nicht der Fall, wird die Einfügeoperation mit einem Fehler **abgebrochen**
- DBMS verhindert Datensätze, deren Fremdschlüsselattribute **beim Einfügen** „ins Leere zeigen“

Tabellenerstellung am Beispiel



```
CREATE TABLE Dozent (  
    Personalnummer INT AUTO_INCREMENT,  
    Name VARCHAR(50) NOT NULL,  
    Telefonnummer VARCHAR(20),  
    EMail VARCHAR(40) NOT NULL,  
    PRIMARY KEY (Personalnummer)  
);
```

Tabellenerstellung am Beispiel

Dozent	
♦Personalnummer	INT
•Name	VARCHAR(50)
◦Telefonnummer	VARCHAR(20)
•E-Mail	VARCHAR(40)

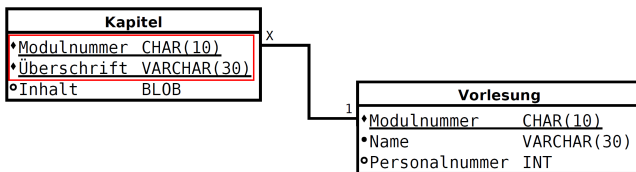
Vorlesung	
♦Modulnummer	CHAR(10)
•Name	VARCHAR(30)
◦Personalnummer	INT

```
CREATE TABLE Vorlesung (  
    Modulnummer      CHAR(10),  
    Name             VARCHAR(30) NOT NULL,  
    Personalnummer   INT NOT NULL,  
    PRIMARY KEY (Modulnummer),  
    FOREIGN KEY (Personalnummer)  
        REFERENCES Dozent(Personalnummer)  
        ON UPDATE CASCADE ON DELETE RESTRICT  
);
```

Beispiel ../Code/010_CREATE_TABLE_Dozent_und_Vorlesung.sql

Tabellenerstellung am Beispiel

Zusammengesetzter Primärschlüssel:



```
CREATE TABLE Kapitel (  
    Modulnummer      CHAR(10),  
    Ueberschrift     VARCHAR(30),  
    Inhalt           BLOB,  
    PRIMARY KEY (Modulnummer, Ueberschrift),  
    FOREIGN KEY (Modulnummer)  
        REFERENCES Vorlesung(Modulnummer)  
        ON UPDATE CASCADE ON DELETE CASCADE  
);
```

Beispiel ../Code/015_CREATE_TABLE_Kapitel.sql

- **SQL** = **S**tructured **Q**uery **L**anguage
- Data Definition Language zur Bearbeitung der **Struktur** der Datenbasis
- **CREATE DATABASE** ... erzeugt **neues Schema** innerhalb der Datenbasis
- **CREATE TABLE** ... erzeugt **neue Tabelle** (=Relation) innerhalb eines Schemas
 - Spalten mit Datentyp
 - Primärschlüssel
 - Fremdschlüssel
- DBMS stellt **referenzielle Integrität** zwischen den Tabellen sicher, wenn Fremdschlüsselbeziehungen definiert wurden

- 1 Einführung
- 2 Entity Relationship Modell
- 3 technisches Modell
- 4 Qualitätskriterien für relationales Modell
- 5 SQL - Übersicht und Data Definition Language
- 6 SQL - Data Definition Language Teil 2**
- 7 SQL - Data Manipulation Language

Wiederholung

6 SQL - Data Definition Language Teil 2

- Löschen von Tabellen und Ändern der Tabellen Struktur
- zirkuläre Fremdschlüsselbeziehungen
- Check Constraint
- reguläre Ausdrücke

Löschen von Tabellen

- Syntax:
`DROP TABLE [IF EXISTS] <tabellenname>;`
- entfernt die Tabellendefinition inkl. aller enthaltenen Tupel

Achtung

Es erfolgt keine Rückfrage, ob die Tabelle gelöscht werden soll!

Achtung 2

Verweist ein Fremdschlüssel auf die zu löschende Tabelle, kann die Tabelle nicht gelöscht werden. → Zuerst muss die Fremdschlüsselbeziehung gelöscht werden.

- Syntax:

TRUNCATE TABLE <tabellenname>;

- entfernt **alle Tupel** einer Tabelle
- Tabellendefinition bleibt erhalten

→ Tabelle nach Ausführung des Statements leer

Achtung

Es erfolgt keine Rückfrage, ob die Tabelle geleert werden soll!

- DBMS intern wird **TRUNCATE** meist durch **DROP** gefolgt von **CREATE** implementiert

Achtung 2

TRUNCATE kann nicht rückgängig gemacht werden

Ändern der Tabellen Struktur

- Syntax:

```
ALTER TABLE <tablename> alter_specification [,  
→ alter_specification ...];
```

- mögliche alter_specifications (Auswahl)

- **RENAME [TO]** <new_table_name> → umbenennen einer Tabelle
- **RENAME COLUMN** old_col_name **TO** new_col_name → Spalte umbenennen
- **ADD COLUMN [IF NOT EXISTS]** <column_name>
column_definition, für column_definition siehe Seite 115
- **DROP COLUMN [IF EXISTS]** <column_name> → löscht Spalte column_name
- **ADD CONSTRAINT [<constraint_name>] PRIMARY KEY**
(<column_names>) → erzeugt Primärschlüssel
- **ADD CONSTRAINT [<constraint_name>] FOREIGN KEY**
foreign_key_specification → erzeugt Fremdschlüssel, für foreign_key_specification siehe Seite 119
- **DROP CONSTRAINT [IF EXISTS]** <constraint_name> → löscht bestehendes Constraint

Achtung

neu erstellte Constraints wirken sich nur auf **zukünftige** Tupel aus.
Bestehende Tupel können ein neues Constraint verletzen!

- Beispiel: Neue Spalten für die Anschrift in der Relation „Dozent“:
USE hans;

```
ALTER TABLE Dozent
  ADD COLUMN IF NOT EXISTS Strasse VARCHAR(30) NOT
    ↪ NULL,
  ADD COLUMN IF NOT EXISTS Hausnummer VARCHAR(5) NOT
    ↪ NULL,
  ADD COLUMN IF NOT EXISTS PLZ CHAR(5) NOT NULL,
  ADD COLUMN IF NOT EXISTS Ort VARCHAR(30) NOT NULL;
```

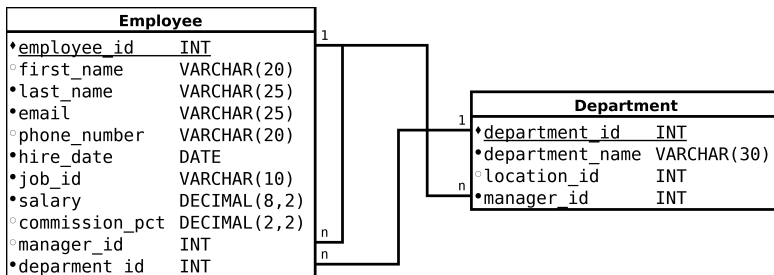
Beispiel ../Code/020_ADD_COLUMN_Dozent.sql

6 SQL - Data Definition Language Teil 2

- Löschen von Tabellen und Ändern der Tabellen Struktur
- zirkuläre Fremdschlüsselbeziehungen
- Check Constraint
- reguläre Ausdrücke

zirkuläre Fremdschlüsselbeziehungen

- **zirkuläre Fremdschlüsselbeziehung:** zwei oder mehr Relationen sind wechselseitig über Fremdschlüsselbeziehungen verknüpft
- Beispiel:



- `employee.department_id` referenziert `department.department_id`
- `department.manager_id` referenziert `employee.employee_id`

- **Problem:** bei der Erstellung der ersten Relation kann die Fremdschlüsselbeziehung zur zweiten Relation noch nicht hergestellt werden
- **Lösung:**
 - die erste Relation wird **zunächst ohne** Fremdschlüsselbeziehung zur zweiten Relation erstellt
 - die zweite Relation wird **mit** Fremdschlüsselbeziehung zur ersten Relation erstellt
 - die ersten Relation wird **nachträglich** um die Fremdschlüsselbeziehung erweitert

Beispiel `../Code/050_zirkulaere_Fremdschluessel_Bezeichnung.sql`

Fremdschlüsselprüfung bei zirkulären Fremdschlüsselbeziehungen

- bei zirkulären Fremdschlüsselbeziehungen kann es zum Einfügen neuer Daten **notwendig** sein, die Prüfung von Fremdschlüsselbeziehungen **temporär zu deaktivieren**
- Deaktivierung der Fremdschlüsselprüfung:
`SET FOREIGN_KEY_CHECKS=0;`
- (Re-)Aktivierung der Fremdschlüsselprüfung:
`SET FOREIGN_KEY_CHECKS=1;`
- unter welchen Voraussetzungen entsteht die Notwendigkeit die Fremdschlüsselprüfung zu deaktivieren?

6 SQL - Data Definition Language Teil 2

- Löschen von Tabellen und Ändern der Tabellen Struktur
- zirkuläre Fremdschlüsselbeziehungen
- **Check Constraint**
- reguläre Ausdrücke

- Constraints definieren **Regeln** für die **Konsistenz** der Datenbasis
- Constraints werden insb. bei der **Ausführung** von **Data Manipulation Language (DML) Statements** durch das DBMS geprüft.
- wird ein Constraint verletzt, wird die Ausführung der Transaktion verweigert, um die Konsistenz der Datenbasis zu wahren
- bereits bekannte Constraints:
 - Primary Key
 - Foreign Key
 - Unique
 - NOT NULL
- darüber hinaus: Definition von **Anwendungsdomänenspezifischen** Constraints möglich

- **Check Constraints** definieren zusätzliche Bedingungen für die Gültigkeit von **neu zu erstellenden** Tupeln
- mögliche Bedingungen (Auswahl)
 - Arithmetische **Vergleiche** ($>$, $<$, $<=$, etc.)
 - Prüfung definierter Wertemengen mittels **IN**
 - Prüfung **regulärer Ausdrücke** (siehe nächstes Kapitel)
 - Aufruf von Funktionen
- werden die Bedingungen des Check Constraints nicht erfüllt, wird das neue Tupel **nicht** in die Relation eingefügt
- Definition als Teil des **CREATE TABLE ...** Statements oder nachträglich mittels **ALTER TABLE ...** möglich
- Syntax:

```
CONSTRAINT <constraint_name> CHECK (<condition>)
```

Check Constraint am Beispiel

- Beispiel 1: Das Gehalt in der „employee“ Relation soll positiv sein:

```
CREATE TABLE employee (  
    ...  
    salary DECIMAL(8, 2) NOT NULL,  
    ...  
    CONSTRAINT SalaryPositive CHECK (salary > 0)  
);
```

- Beispiel 2: Wochentag nur aus der Wertemenge „Mo“, „Di“, „Mi“, „Do“ und „Fr“

```
CREATE TABLE Raumbelegung(  
    ...  
    Wochentag CHAR(2),  
    ...  
    CONSTRAINT WochentagPattern CHECK (Wochentag IN  
        ↪ ('Mo', 'Di', 'Mi', 'Do', 'Fr'))  
);
```

6 SQL - Data Definition Language Teil 2

- Löschen von Tabellen und Ändern der Tabellen Struktur
- zirkuläre Fremdschlüsselbeziehungen
- Check Constraint
- reguläre Ausdrücke

- **reguläre Ausdrücke** definieren eine „Schablone“ für Textinformationen
- reguläre Ausdrücke sind selbst **Zeichenketten**
- die im regulären Ausdruck enthaltenen Zeichen, können jeweils eine **eigene Bedeutung** haben - siehe nächste Folie
- Textinformationen können geprüft werden, ob sie einem regulären Ausdruck entsprechen oder nicht
- beim Check Constraint mittels Schlüsselwort **RLIKE**

- Zeichen ohne Sonderbedeutung müssen 1:1 in der Textinformation enthalten sein
- **Platzhalter** - repräsentieren **ein** Zeichen in der zu prüfenden Textinformation:
 - „.“ → ein beliebiges Zeichen
 - „[a-z]“ → ein Kleinbuchstabe von a - z
 - „[A-Z]“ → ein Großbuchstabe von A - Z
 - „[0-9]“ → eine Ziffer von 0 - 9
 - **Kombinationen** sind möglich z.B: „[a-z0-7]“ → ein Kleinbuchstabe oder eine Ziffer von 0 bis 7
- **benannte Zeichenklassen**
 - „[:alnum:]“ → alphanumerisches Zeichen
 - „[:digit:]“ → Ziffer
 - „[:xdigit:]“ → hexadezimale Ziffer
 - „[:blank:]“ → Whitespace (Leerzeichen, Tabulator, Zeilenumbruch, ...)

- **Wiederholungs-** Definitionen des vorhergehenden Platzhalters
 - ohne weitere Angabe $\rightarrow$ Platzhalter muss exakt einmal vorkommen
 - „*“ $\rightarrow$ beliebige viele Vorkommen inkl. 0
 - „+“ $\rightarrow$ mindestens ein Vorkommen
 - „?“ $\rightarrow$ 0 oder ein Vorkommen
 - „{m}“ $\rightarrow$ genau m Vorkommen
 - „{m,n}“ $\rightarrow$ mindestens m , höchstens n Vorkommen
- **Gruppierung** von Platzhaltern mittels „()“
- Definition von **alternativen Platzhaltern** mittels „(Option1|Option2)“
- **Escaping** von Zeichen mit Sonderbedeutung mittels „\“
- „^“ $\rightarrow$ Beginn des zu prüfenden Textes
- „\$“ $\rightarrow$ Ende des zu prüfenden Textes

$\rightarrow$ weitere Informationen unter
<https://mariadb.com/kb/en/regular-expressions-overview/>

- Definition eines einfachen E-Mail Adressformats bestehend aus folgenden Bestandteilen:
 - mindestens ein beliebiges Zeichen
 - „@“ Symbol
 - mindestens ein beliebiges Zeichen
 - „.“ Symbol
 - mindestens ein beliebiges Zeichen
- resultierender regulärer Ausdruck: `.+@.+\. .+`
- als Teil eines **CHECK CONSTRAINT**:
CHECK EMailPattern **CHECK** (EMail RLIKE ' .+@.+\. .+')

- Definition eines flexiblen Telefonnummernformats
 - Durchwahl mit 3 bis 10 Ziffern Länge, erste Ziffer nicht 0
 - optional vorangestellte Vorwahl
 - optional mittels „/“ von Durchwahl getrennt
 - 3- oder 4- stellige Ortsvorwahl, erste Ziffer nicht 0
 - vorausgehende „0“ oder
 - optional durch Leerzeichen getrennte Landesvorwahl im Format +49 oder 0049
- resultierender regulärer Ausdruck:
$$^(((\backslash+|00)[1-9][0-9]? \ ?|0)[1-9][0-9]\{2,3\}/?)?[1-9][0-9]\{2,9\}\$$$

Erstellung des Beispiel Schemas

Beispiel `../Code/099_CREATE_DB_HANS.sql`

- Löschen einer Tabelle mittels **DROP TABLE ...**
 - Ändern der Tabellenstruktur mittels **ALTER TABLE ...**
 - **zirkuläre** Fremdschlüsselbeziehungen erfordern **schrittweise Erstellung** der Tabellen und Fremdschlüsseldefinitionen
 - **Check Constraints** ermöglichen Anwendungsdomänenspezifische Bedingungen für neue Tupel in Tabellen
 - **reguläre Ausdrücke** ermöglichen die Definition von Schablonen für textuelle Information
 - Platzhalter
 - Zeichenklassen
 - Wiederholung und Gruppierung
 - Alternativen
- Verwendung z.B. in Check Constraints möglich

- 1 Einführung
- 2 Entity Relationship Modell
- 3 technisches Modell
- 4 Qualitätskriterien für relationales Modell
- 5 SQL - Übersicht und Data Definition Language
- 6 SQL - Data Definition Language Teil 2
- 7 SQL - Data Manipulation Language**

Wiederholung

7 SQL - Data Manipulation Language

- Einfügen von Daten
- Aktualisierung von Daten
- Insert bei zirkulären Fremdschlüsselbeziehungen
- Löschen von Daten

- **Einfügen** von Daten mittels **INSERT** Statement
 - Drei Varianten der Syntax für **unterschiedliche Anwendungsfälle**
 - mit „**VALUES**“ Schlüsselwort
 - mit „**SET**“ Schlüsselwort
 - **SELECT** zum bedingten Kopieren von Daten aus anderen Tabellen
 - **INSERT** setzt vorhandene Tabelle voraus
- erst nach dem jeweiligen **CREATE TABLE** ... Statement möglich

- folgende Daten sollen in die Dozenten Relation eingefügt werden:

<u>Pers.Nr.</u>	Name	Telefonnummer	E-Mail
100	Schneider	0123/45678100	Schneider@hs-ansbach.de
200	Fischer	<leer>	Fischer@hs-ansbach.de
300	Weber	0123/45678300	Weber@hs-ansbach.de
400	Wagner	0123/45678400	Wagner@hs-ansbach.de
500	Becker	0123/45678500	Becker@hs-ansbach.de
600	Schmidt	0123/45678600	Schmidt@hs-ansbach.de

Einfügen von Daten mit VALUES Schlüsselwort

- Syntax:

```
INSERT [INTO] <tabellenname> [(<spaltenname1,  
→ spaltenname2, ...)] VALUES (<wert_spalte1,  
→ wert_spalte2, ...>), [(<wert_spalte1, wert_spalte2,  
→ ...>), ...];
```

- **Reihenfolge der Spaltennamen** muss zur Reihenfolge der Werte passen
- Liste der Spaltennamen kann weggelassen werden, wenn für **alle Spalten** ein Wert angegeben wird
- Einfügen **mehrerer** Tupel **gleichzeitig** möglich
- Beispiel:

```
INSERT INTO Dozent(Personalnummer, Name, Telefonnummer,  
→ EMail) VALUES (100, 'Schneider', '0123/45678100',  
→ 'Schneider@hs-ansbach.de');
```

[Beispiel ../Code/100_INSERT_Varianten.sql](#)

Einfügen von Daten mit SET Schlüsselwort

- Syntax:

```
INSERT [INTO] <tabellenname> SET  
  → <spaltenname1>=<wert_spalte1> [,  
  → <spaltenname2>=<wert_spalte2>, ...];
```

- Spaltennamen müssen explizit angegeben werden
- nur **ein Tupel** pro Statement möglich
- ähnelt der Syntax des **UPDATE** Statements (siehe folgendes Unterkapitel)
- Beispiel:

```
INSERT INTO Dozent SET Personalnummer = 600, Name =  
  → 'Schmidt', Telefonnummer = '0123/45678600', Email =  
  → 'Schmidt@hs-ansbach.de';
```

Beispiel ../Code/100_INSERT_Varianten.sql

- Syntax:

```
INSERT [INTO] <tabellenname> [(<spaltenname1,  
→ spaltenname2, ...)] SELECT ...;
```

- Ziel: Daten werden aus **bestehenden Tabellen** selektiert und in die aktuelle Tabelle eingefügt. (**SELECT** siehe Kapitel zu SQL DQL)
- **Achtung:** Erstellung von Kopien **widerspricht** dem Ziel der Redundanzfreiheit
- → kommt meist nur im Rahmen von Änderungen der Schemastruktur beim Umzug bestehender Daten aus der alten in die neue Struktur zum Einsatz
- Liste der Spaltennamen kann weggelassen werden, wenn für **alle Spalten** ein Wert aus den bestehenden Tabellen selektiert wird

7 SQL - Data Manipulation Language

- Einfügen von Daten
- Aktualisierung von Daten
- Insert bei zirkulären Fremdschlüsselbeziehungen
- Löschen von Daten

- Syntax:

UPDATE <tabellenname> **SET**

→ <spaltenname1>=<neuer_wert_spalte1> [,
→ <spaltenname2>=<neuer_wert_spalte2>, ...] [**WHERE**
→ <where_condition>]

- angelehnt an Variante 2 des **INSERT** Statements
- es müssen nur die Spalten angegeben werden, deren Wert sich ändern soll
- **WHERE** steuert, **welche** Tupel aktualisiert werden sollen

Achtung

ohne where_condition werden die jeweiligen Spalten **aller existierenden** Tupel mit den neuen Werten überschrieben!

- Die Telefonnummer des Dozenten Fischer soll aktualisiert werden

```
UPDATE Dozent SET Telefonnummer = '0123/45678200' WHERE  
↪ Personalnummer = 200;
```

- Hinweis: es werden **alle Tupel**, für die die **Bedingung** im **WHERE** Statement zutrifft aktualisiert

→ Soll **ein einziges** Tupel aktualisiert werden, sollte dieses über **seinen Primärschlüssel Wert** ausgewählt werden

Beispiel [../Code/110_UPDATE.sql](#)

- bei **allen** **INSERT** oder **UPDATE** Operationen werden die für die **jeweilige Relation** relevanten **Constraints** geprüft
- wird ein Constraint verletzt, wird die Operation mit einem Fehler **abgebrochen**

→ **Konsistenz** der Datenbasis **bleibt erhalten**

Beispiel `../Code/120_Constraint_Violation_Primary_Key.sql`

Beispiel `../Code/121_Constraint_Violation_Foreign_Key.sql`

Beispiel `../Code/125_Constraint_Violation_Check_Constraint.sql`

- **UPDATE** Statements führen zur Auswertung der **referenziellen Optionen** der **ON UPDATE** Anweisungen von Fremdschlüsseldefinitionen
- **CASCADE** → Änderung des Primärschlüssel Wertes in der referenzierten Tabelle führt zur Änderung der referenzierenden Fremdschlüsselwerte

Beispiel `../Code/130_referenzielle Optionen_UPDATE_CASCADE.sql`

- **RESTRICT** → Änderung des Primärschlüssel Wertes in der referenzierten Tabelle nur zulässig, falls kein referenzierender Fremdschlüsselwert existiert.

Beispiel `../Code/131_referenzielle Optionen_UPDATE_RESTRICT.sql`

INSERT ... ON DUPLICATE KEY UPDATE ...

- sollen große Datenmengen in eine bereits befüllte Datenbasis importiert werden, können Duplikate auftreten
- **INSERT** wird mit „duplicate key“ Fehler abgebrochen um Eindeutigkeit zu gewährleisten
- einige DBMS unterstützen „Insert or Update“ Operationen
- bei MySQL/MariaDB:
INSERT ... ON DUPLICATE KEY UPDATE ...
- Syntax:
INSERT [INTO] <tabellenname> ...
ON DUPLICATE KEY UPDATE
 ↪ <spaltenname1>=<neuer_wert_spalte1> [,
 ↪ <spaltenname2>=<neuer_wert_spalte2>, ...]
- **ON DUPLICATE KEY UPDATE** Erweiterung existiert für alle drei **INSERT** Varianten

- füge neuen Dozent mit Personalnummer 700 ein, falls die Personalnummer 700 bereits existiert, aktualisiere die relevanten Spalten:

```
INSERT INTO Dozent SET Personalnummer=700, Name='Höft',  
  ↳ Telefonnummer='0123/45678800',  
  ↳ EMail='Hoeft@hs-ansbach.de'  
ON DUPLICATE KEY UPDATE Name='Höft',  
  ↳ Telefonnummer='0123/45678800',  
  ↳ EMail='Hoeft@hs-ansbach.de';
```

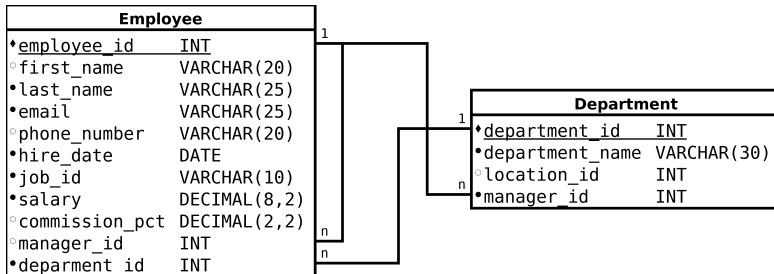
Beispiel ../Code/140_INSERT_ON_DUPLICATE_KEY_UPDATE.sql

7 SQL - Data Manipulation Language

- Einfügen von Daten
- Aktualisierung von Daten
- Insert bei zirkulären Fremdschlüsselbeziehungen
- Löschen von Daten

Insert bei zirkulären Fremdschlüsselbeziehungen

- gegebenes Schema:



- employee.department_id referenziert department.department_id
- department.manager_id referenziert employee.employee_id
- employee.department_id und department.manager_id sind jeweils notwendig (**NOT NULL**)
- **Henne-Ei Problem**

Temporäre Deaktivierung der Fremdschlüsselprüfung

- Lösung: Fremdschlüsselprüfung muss **vorübergehend deaktiviert** werden
- Deaktivierung der Fremdschlüsselprüfung:
`SET FOREIGN_KEY_CHECKS=0;`
- (Re-)Aktivierung der Fremdschlüsselprüfung:
`SET FOREIGN_KEY_CHECKS=1;`

Beispiel

`../Code/145_INSERT_bei_zirkulaeren_Fremdschlüsselbeziehungen.sql`

Achtung

keine nachträgliche Überprüfung der Fremdschlüsselbeziehungen beim Wiedereinschalten der Fremdschlüsselprüfung!

7 SQL - Data Manipulation Language

- Einfügen von Daten
- Aktualisierung von Daten
- Insert bei zirkulären Fremdschlüsselbeziehungen
- Löschen von Daten

- Syntax:

```
DELETE FROM <tabellenname> [WHERE where_condition]  
    ↪ [ORDER BY ...] [LIMIT row_count]
```

- **WHERE** wählt zu löschende Datensätze aus
- **LIMIT** beschränkt die Anzahl der zu löschenden Einträge auf **row_count**
- **ORDER BY** sortiert die Tupel vor dem Löschen nach dem angegebenen Kriterium → nur sinnvoll in Kombination mit **LIMIT**

Achtung

Ohne **WHERE** Klausel werden **alle** Tupel einer Relation gelöscht!

- Dozent mit Personalnummer 500 soll gelöscht werden

```
DELETE FROM Dozent WHERE Personalnummer = 500;
```

Beispiel [../Code/150_DELETE.sql](#)

Wirkung referenzieller Optionen

- **DELETE** Statements führen zur Auswertung der **referenziellen Optionen** der **ON DELETE** Anweisungen von Fremdschlüsseldefinitionen
- mögliche referenzielle Optionen für **DELETE** Statement: **CASCADE**, **RESTRICT**
- **CASCADE** → Löschen eines Tupels in der referenzierten Tabelle führt zum Löschen der referenzierenden Tupel
Beispiel [../Code/155_referenzielle Optionen_DELETE_CASCADE.sql](#)
- Hinweis: sind mehrere Relationen mittels Fremdschlüsselbeziehungen und **ON DELETE CASCADE** verkettet, werden auch die referenzierenden Tupel in den **verketteten Relationen** gelöscht!
- **RESTRICT** → Löschen eines Tupels in der referenzierten Tabelle wird verhindert, falls Tupel in der referenzierenden Relation existieren, die auf das zu löschende Tupel verweisen
Beispiel [../Code/156_referenzielle Optionen_DELETE_RESTRICT.sql](#)

Achtung

DDL Statements bearbeiten die **Struktur** der Daten

DML Statements adressieren die Daten **innerhalb der Relationen**

- DDL Statements: **CREATE**, **ALTER** und **DROP**
- DML Statements: **INSERT**, **UPDATE** und **DELETE**

- **Einfügen** von Daten in bestehende Relationen mittels **INSERT** ...
- drei **INSERT Varianten** je nach Anwendungsfall
- Steuerung der **Fremdschlüsselprüfung** mittels **SET FOREIGN_KEY_CHECKS=...**
- **Aktualisieren** von Daten mittels **UPDATE** ...
- „Insert or Update“ mittels **INSERT ... ON DUPLICATE KEY UPDATE ...**
- **Löschen** von Daten mittels **DELETE** ...
- **UPDATE** und **DELETE** führen zur Auswertung der **referenziellen Optionen** von Fremdschlüsseldefinitionen

- 8 SQL - Data Query Language
- 9 SQL - Data Query Language - Teil 2
- 10 SQL - Mengenoperationen und Views

Wiederholung

- Dient zum **Abfragen von Daten** aus dem jeweiligen Schema
- Festlegung **welche Attribute** ausgegeben werden sollen
- Verknüpfung von **mehreren Relationen**
- **Filtern** von Informationen nach beliebigen Kriterien
- **Sortieren** der Informationen nach beliebigen Kriterien
- Zusammenfassen von Daten zu **Gruppen** inkl. Aggregationsfunktionen
- **Mengenoperationen** (z.B. Vereinigungsmenge, Schnittmenge, etc.)
- wichtigster Bestandteil der **Data Query Language** (DQL): **SELECT** Statement

- 8 SQL - Data Query Language
 - Das SELECT Statement
 - Operatoren und Single Row Functions
 - Filterung von Ergebnissen
 - Beispiel

Einführung

- einfachste Form:

```
SELECT * FROM <tabellenname>;
```

→ liefert **alle** Attribute **aller** Tupel in der Relation <tabellenname>

- Beispiel:

```
SELECT * FROM dozent;
```

- Erwartetes Ergebnis:

<u>Pers.Nr.</u>	Name	Telefonnummer	EMail
100	Schneider	0123/45678100	Schneider@hs-ansbach.de
200	Fischer	0123/45678200	Fischer@hs-ansbach.de
300	Weber	0123/45678300	Weber@hs-ansbach.de
400	Wagner	0123/45678400	Wagner@hs-ansbach.de
500	Becker	0123/45678500	Becker@hs-ansbach.de
600	Schmidt	0123/45678600	Schmidt@hs-ansbach.de

[Beispiel ../Code/200_SELECT_Einfuehrung.sql](#)

- Spaltennamen des Ergebnisses = Attributname innerhalb der Relation

Das SELECT Statement

- Syntax (vereinfacht):

```
SELECT [DISTINCT] select_expr [, select_expr ...]  
FROM table_references  
[WHERE where_condition]  
[GROUP BY <spaltenname1> [, <spaltenname2> ...]]  
[HAVING where_condition]  
[ORDER BY <spaltenname1> [ASC | DESC] [,  
↪ <spaltenname2> [ASC | DESC] ...]]  
[LIMIT row_count];
```

- viele Teile sind **optional**
- **Reihenfolge** der optionalen Bestandteile muss eingehalten werden!

die FROM Klausel

- **FROM** Klausel definiert die **Datenquelle**, aus der Informationen extrahiert werden sollen
- **FROM** Klausel **muss** angegeben werden
- mögliche Optionen für table_references:
 - **eine** Relation in der Form <schemaname>.<relationsname>
 - Verknüpfung **mehrerer** Relationen (→ siehe **JOIN**)
 - eine **Unterabfrage**
- Beispiel: Relation Vorlesung aus dem Schema hans als Datenquelle
SELECT ... **FROM** hans.Vorlesung;
- Tipp: wird vor dem **SELECT** Statement ein USE <schemaname> Statement verwendet, kann der <schemaname> in der **FROM** Klausel entfallen
USE hans;
SELECT ... **FROM** Vorlesung;

Sonderfall: FROM DUAL

- Sonderfall wenn **keine Datenmenge** benötigt wird: Schlüsselwort DUAL
- Anwendungsfälle:
 - Berechnungen innerhalb eines SQL Skripts
 - Zugriff auf global gültige Informationen (Datum, Uhrzeit, ...)
- Beispiele:

```
SELECT 1 + 1 FROM DUAL;
```

```
SELECT curdate() FROM DUAL;
```

Beispiel [../Code/209_SELECT_FROM_DUAL.sql](#)

Die select_expression

- Die select_expr definiert **welche Informationen** aus der Datenquelle extrahiert werden sollen
- Wichtig: jede select_expr definiert **eine Spalte** des Ergebnisses
- Optionen:
 - *: **alle Attribute** der Datenquelle
 - <spaltenname1> [, <spaltenname2>, ...]: **konkrete** Attribute

- Beispiel:

SELECT Modulnummer, Name **FROM** Vorlesung;

- liefert:

Modulnummer	Name
KIK-Engl	technisches Englisch
KIK-Info1	Grundlagen der Informatik
KIK-Mathe1	Mathematik für Informatiker
WIN-Mathe1	Mathematik für Wirtschaftsing.
WIN-ProdT	Produktionstechnik

- Die Informationen können mittels **Operatoren** und **Funktionen** vor der Ausgabe weiterverarbeitet werden

- 8 SQL - Data Query Language
 - Das SELECT Statement
 - Operatoren und Single Row Functions
 - Filterung von Ergebnissen
 - Beispiel

- **Arithmetische Operatoren:**

- +, -, *, /
- ganzzahlige Division: DIV
- Modulooperator: %

- **logische Operatoren**

- ! logische Invertierung (=Verneinung)
- && bzw. **AND** logische „AND“ Verknüpfung
- || bzw. **OR** logische „OR“ Verknüpfung
- XOR logische „XOR“ Verknüpfung

- **Bitweise Operatoren**

- ~ Bitweise Invertierung
- & Bitweise „AND“ Verknüpfung
- | Bitweise „OR“ Verknüpfung
- ^ Bitweise „XOR“ Verknüpfung

- **Klammerung** von Teilausdrücken wird unterstützt

Single Row Functions

- **Single Row Functions** verarbeiten Daten aus der Datenquelle zeilenweise
- Anwendungsfälle:
 - Verarbeitung / Manipulation der Daten
 - Konvertieren des Datentyps
- Beispiele:
 - Case Conversion (**LOWER**, **UPPER**, INITCAP)
 - String Manipulation
 - Datumsfunktionen
 - Behandlung von **NULL** Werten
 - Arithmetische Funktionen

Achtung

Verfügbare Single Row Functions hängen von eingesetztem DBMS ab!

String Manipulation

- CONCAT → Aneinanderreihen von Strings:
 - → CONCAT('Hello', 'World') liefert 'HelloWorld'
- SUBSTR → Extrahiert Teile eines Strings:
 - → SUBSTR('HS Ansbach', 7, 4) liefert 'bach'
- LENGTH → Länge eines Strings:
 - → LENGTH('HS Ansbach') liefert 10
- INSTR → Sucht erste Position eines Zeichens / Substrings:
 - → INSTR('foobarbar', 'bar') liefert 4
- LPAD bzw. RPAD → links- bzw. rechtsseitiges Auffüllen von Strings:
 - → LPAD(2000, 10, '*') liefert *****2000
- REPLACE → Suchen und Ersetzen in Strings
 - →
REPLACE('SQL ist langweilig', 'langweilig', 'spannend')
liefert 'SQL ist spannend'

Beispiel [../Code/210_SELECT_Single_Row_Functions.sql](#)

- NOW → liefert aktuellen Zeitstempel
 - NOW() liefert z.B: 2023-05-30 12:21:44
 - STR_TO_DATE → konvertiert Datumsstring in DATETIME Datentyp
 - STR_TO_DATE('12:25 30.05.2023', '%H:%i %d.%m.%Y') liefert 2023-05-30 12:25:00
 - DATE_FORMAT → konvertiert DATETIME in angegebenes Format
 - DATE_FORMAT(NOW(), '%H:%i %d.%m.%Y') liefert z.B: 12:27 30.05.2023
 - DATEDIFF → berechnet Anzahl an Tagen zwischen zwei Dati
 - DATEDIFF('2023-05-30', '2023-03-15') liefert 76
- Beispiel [../Code/211_SELECT_Datumsfunktionen.sql](#)

Umgang mit NULL Werten

- IFNULL bzw. NVL → liefert alternativen Wert, falls erster Parameter **NULL**
 - IFNULL('sinnvoller Wert', 'N/A') liefert 'sinnvoller Wert'
 - IFNULL(**NULL**, 'N/A') liefert 'N/A', da erster Parameter **NULL**
- NVL2 → liefert dritten Parameter wenn erster Parameter **NULL**, sonst zweiten Parameter
 - NVL2('sinnvoller Wert', 1, 2) liefert 1
 - NVL2(**NULL**, 1, 2) liefert 2
- **NULLIF** → liefert **NULL** wenn beide Parameter identisch, sonst ersten Parameter
 - **NULLIF**(1, 1) liefert **NULL**
 - **NULLIF**(1, 2) liefert 1

Beispiel ../Code/212_SELECT_Umgang_mit_NULL.sql

- **ROUND** → Runden von Zahlenwerten auf definierte Anzahl von Nachkommastellen
 - **ROUND**(42.276, 2) liefert 42.28
 - **TRUNCATE** → Abrunden / Abschneiden von Zahlenwerten auf definierte Anzahl von Nachkommastellen
 - **TRUNCATE**(42.276, 2) liefert 42.27
 - **ABS** → Betrag einer Zahl
 - **ABS**(-42) liefert 42
 - **SIN, COS, TAN, ASIN, ACOS, ATAN, ...** → trigonometrische Funktionen
 - **COS**(0) liefert 1
- Beispiel [../Code/213_SELECT_arithmetische_Funktionen.sql](#)

Fallunterscheidung mittels CASE

- **CASE** Operator ermöglicht **Fallunterscheidung** analog zu **if** Statements imperativer Programmiersprachen

- Syntax:

```
CASE WHEN <condition> THEN result [WHEN <condition>  
  ↪ THEN result ...] [ELSE result] END
```

- liefert **result**, wenn die zugehörige Bedingung **TRUE** ist.
- Beispiel:

```
SELECT CASE WHEN 1 >= 0 THEN 'positiv' ELSE 'negativ'  
  ↪ END FROM DUAL
```

- Beispiel 2:

```
SELECT CASE WHEN Uhrzeit < '12:00' THEN 'Vormittag'  
  ↪ ELSE 'Nachmittag' END FROM Raumbelegung;
```

Beispiel [../Code/214_SELECT_CASE_Operator.sql](#)

Spaltenalias

- die **Spaltenüberschriften** des Ergebnis entsprechen den `select_expr`
- Anpassung der Spaltenüberschrift mittels **Spaltenalias** möglich
- Syntax:

`<ursprüngliche select_expr> AS '<Spaltenalias>'`

- Beispiel:

`SELECT Modulnummer AS 'Nr', Name AS 'Fach' FROM Vorlesung;`

- liefert:

Nr	Fach
KIK-Engl	technisches Englisch
KIK-Info1	Grundlagen der Informatik
KIK-Mathe1	Mathematik für Informatiker
WIN-Mathe1	Mathematik für Wirtschaftsing.
WIN-ProdT	Produktionstechnik

→ v.a. bei **komplexen** `select_expr` sinnvoll

DISTINCT

- **Duplikate** im Ergebnis eines **SELECT** Statements möglich
- mittels **DISTINCT** können Duplikate aus der **Ergebnismenge** **herausgefiltert** werden:

```
SELECT DISTINCT ...;
```

Achtung

die Duplikatfilterung wirkt auf die **Gesamtheit der Attribute**. Es erfolgt keine Filterung für einzelne Attribute der Ergebnismenge.

Achtung 2

Es werden nur Duplikate aus der **Ergebnismenge** entfernt. Das Statement hat keine Auswirkung auf die im Schema gespeicherten Daten!

DISTINCT am Beispiel

- gegeben seien folgende Tupel in der Tabelle „Studierende“

<u>Matrikelnummer</u>	Name	Studiengang
815	Müller	WIT
4711	Meyer	WIG
4714	Meyer	WIN
...	...	...

SELECT Name **FROM**
↪ Studierende;

Name
Müller
Meyer
Meyer
...

SELECT DISTINCT Name **FROM**
↪ Studierende;

Name
Müller
Meyer
...

Beispiel [../Code/220_SELECT_DISTINCT.sql](#)

- 8 SQL - Data Query Language
 - Das SELECT Statement
 - Operatoren und Single Row Functions
 - **Filterung von Ergebnissen**
 - Beispiel

Filterung von Ergebnissen

- **WHERE** Klausel **filtert** das Ergebnis nach **definierbaren Kriterien**
→ nur Tupel, die das definierte **Kriterium erfüllen**, werden in das Ergebnis übernommen

- Vergleichsoperatoren

Operator	Bedeutung
=	ist gleich (NULL wenn min. ein Operand NULL)
<=>	ist gleich („NULL safe“ → 1 wenn beide Operanden NULL ; 0 wenn eine der Operanden NULL)
<	kleiner
>	größer
<=	kleiner gleich
>=	größer gleich

- Operator zum Vergleich mit **Wertemenge**:
... **IN** (<wert1>, <wert2>, ...)
- Prüfung von **NULL** mittels ... **IS** [**NOT**] **NULL**

- Prüfung auf **exakte Übereinstimmung** mittels = Operator
- **LIKE** Operator ermöglicht Stringvergleich mit einfacher **Wildcard Unterstützung**
- Syntax:
`<string> [NOT] LIKE <pattern>`
- `<string>` ist der zu vergleichende String
- `<pattern>` definiert Schablone; unterstützte Wildcards:
 - % → beliebig viele beliebige Zeichen
 - _ → genau ein beliebiges Zeichen
- **RLIKE** Operator ermöglicht Stringvergleich basierend auf regulären Ausdrücken

- Verknüpfung **mehrerer** Filterkriterien über **logische Operatoren**
 - ! logische Invertierung (=Verneinung)
 - && bzw. **AND** logische „AND“ Verknüpfung
 - || bzw. **OR** logische „OR“ Verknüpfung
 - XOR logische „XOR“ Verknüpfung
- **Klammerung** von Teilausdrücken wird unterstützt

- 8 SQL - Data Query Language
 - Das SELECT Statement
 - Operatoren und Single Row Functions
 - Filterung von Ergebnissen
 - Beispiel

Beispiel: Ausgangssituation

- Datenquelle: Relation „pruefung“ des Schemas „hans“
- Auszugebende Informationen:
 - Name der Aufsichtsperson
 - Datum der Prüfung
 - Gebäudenummer
 - „Wintersemester“ oder „Sommersemester“
- nur für Aufsichtspersonen mit Namen „Werner“ oder deren Name mit „Wag“ beginnt
- Datenbestand in der Relation „pruefung“:

<u>ID</u>	Aufsicht	Zeitpunkt	Raum	Semester
10000	'Wagner'	'2023-01-23 11:00'	'50.1.02'	'WiSe2023'
10001	'Wagner'	'2022-07-23 08:00'	'50.1.02'	'SoSe2023'
11001	'Weber'	'2023-01-23 14:00'	'50.1.02'	'WiSe2023'
11002	'Wagenmacher'	'2023-01-27 08:00'	'50.2.21'	'WiSe2023'
11003	'Werner'	'2023-01-22 08:00'	'92.1.24'	'WiSe2023'

Beispiel: SELECT Statement

SELECT

```
Aufsicht,  
DATE_FORMAT(Zeitpunkt, '%d.%m.%Y') AS Datum,  
SUBSTR(Raum, 1, 2) AS Gebäude,  
CASE  
    WHEN SUBSTR(Semester,1,4) = 'WiSe' THEN  
        ↪ 'Wintersemester'  
    WHEN SUBSTR(Semester,1,4) = 'SoSe' THEN  
        ↪ 'Sommersemester'  
    ELSE 'unbekannt'  
END AS Semester  
FROM hans.pruefung  
WHERE Aufsicht LIKE 'Wag%' OR Aufsicht = 'Werner';
```

Beispiel ../Code/230_SELECT_Beispiel.sql

Beispiel: Ergebnis

Aufsicht	Datum	Gebäude	Semester
'Wagner'	'23.01.2023'	'50'	'Wintersemester'
'Wagner'	'23.07.2022'	'50'	'Sommersemester'
'Wagenmacher'	'27.01.2023'	'50'	'Wintersemester'
'Werner'	'22.01.2023'	'92'	'Wintersemester'

- **SELECT** Statement zur **Auswahl von Information** aus der Datenbasis
- Auswahl der **Datenquelle** mittels **FROM** Klausel
- Definition der **Ergebnisspalten** mittels `select_expr`
 - **Spaltenalias** zur Benennung komplexer `select_expr`
 - **DISTINCT** zur **Duplikatfilterung**
- Verarbeitung der Daten mittels **Operatoren** und **Single Row Functions**
 - arithmetische, logische, bitweise Operatoren
 - Stringmanipulation
 - Datumsfunktionen
- **Filterung** der Ergebnisse mittels **WHERE** Klausel

- 8 SQL - Data Query Language
- 9 SQL - Data Query Language - Teil 2
- 10 SQL - Mengenoperationen und Views

Wiederholung

9 SQL - Data Query Language - Teil 2

- Sortieren und Limitieren der Ergebnismenge
- Gruppierung und Aggregatfunktionen
- Verknüpfung mehrerer Relationen mittels Join

Sortieren der Ergebnismenge

- **ORDER BY** Klausel ermöglicht das Sortieren der Ergebnismenge
- Syntax:
... **ORDER BY** <Sortierkriterium1> [**ASC** | **DESC**] [,
↪ <Sortierkriterium2> [**ASC** | **DESC**] ...]
- mögliche **Sortierkriterien**:
 - Spalten**namen** aus Datenquelle
 - Spalten**alias**
- Verwendung von **Operatoren** und **Funktionen** in Sortierkriterien möglich
- Definition der Sortier**reihenfolge**:
 - **ASC**: aufsteigend (Default)
 - **DESC**: absteigend

Beispiel `../Code/250_SELECT_ORDER_BY.sql`

Limitierung der Ergebnismenge

- **LIMIT** Klausel definiert **maximale Anzahl** an Tupeln in der Ergebnismenge
 - Syntax:
... **LIMIT** [**offset** ,] rowcount
 - rowcount definiert die maximale Anzahl an Tupeln
 - **offset** (optional) definiert, ab welchem Tupel die Ergebnismenge beginnen soll
- Achtung: Zählung beginnt bei 0!
- Hinweis: die Limitierung erfolgt **nach** der Sortierung der Ergebnismenge

Beispiel [../Code/255_SELECT_LIMIT.sql](#)

9 SQL - Data Query Language - Teil 2

- Sortieren und Limitieren der Ergebnismenge
- Gruppierung und Aggregatfunktionen
- Verknüpfung mehrerer Relationen mittels Join

Gruppierung von Tupeln

- Die **GROUP BY** Klausel erlaubt die Zusammenfassung von Tupeln der Ergebnismenge zu Gruppen

- Syntax:

```
... GROUP BY <Gruppierkriterium1> [,  
  ↪ <Gruppierkriterium2> ...]
```

- Tupel deren Werte für **alle Gruppierkriterien** übereinstimmen, werden zu **einer Gruppe** zusammengefasst

→ **ein** neues Tupel, das die komplette Gruppe repräsentiert

- **Aggregatfunktionen** zur Zusammenfassung der Attribute, die kein Gruppierkriterium darstellen notwendig

Beispiel [../Code/270_SELECT_GROUP_BY.sql](#)

- **Aggregatfunktionen** verarbeiten die Attribute **mehrerer** Tupel und liefern **ein** Ergebnis

→ = Unterschied zu Single Row Functions

- verfügbare Aggregatfunktionen:
 - **COUNT** liefert **Anzahl an Elementen** in der Gruppe
 - **COUNT DISTINCT** liefert **Anzahl unterschiedlicher Attributwerte** in der Gruppe
 - **GROUP_CONCAT** liefert **Komma separierte Liste** aller Attributwerte
 - **MIN** bzw. **MAX** liefert **minimalen** bzw. **maximalen** Zahlenwert der Attributwerte in der Gruppe
 - **SUM** liefert **Summe** der Attributwerte in der Gruppe
 - **AVG** liefert **Durchschnitt** der Attributwerte in der Gruppe
 - **STDDEV** liefert **Standardabweichung** der Attributwerte in der Gruppe
 - **VARIANCE** liefert **Varianz** der Attributwerte in der Gruppe

Beispiel [../Code/271_SELECT_Aggregatfunktionen.sql](#)

Gruppierung und Select Expressions

- Wird die **GROUP BY** Klausel verwendet, müssen die **Select Expressions eingeschränkt** werden
- erlaubte Select Expressions:
 - **Gruppierungskriterien**
 - mittels **Aggregatfunktionen** verarbeitete Attribute von Tupeln aus der Datenquelle

Achtung

Attribute von Tupeln aus der Datenquelle, die nicht mittels Aggregatfunktion zusammengefasst sind, sollten nicht verwendet werden!

Beispiel [../Code/272_SELECT_GROUP_BY_ohne_Aggregatfunktion.sql](#)

- werden die Attributwerte nicht mittels Aggregatfunktion zusammengefasst, ist das Ergebnis **nicht definiert**
- **ONLY_FULL_GROUP_BY** Modus **erzwingt** die Nutzung gültiger Select Expressions

- bei **mehreren Gruppierungskriterien** → nur Tupel bei denen **alle Gruppierungskriterien** übereinstimmen werden zu Gruppen zusammengefasst
- Beispiel: Gruppierung der employee Tabelle nach Abteilung **und** Manager

Beispiel `../Code/273_SELECT_GROUP_BY_mehrere_Kriterien.sql`

- durch **GROUP BY** erzeugte Gruppen können mittels **HAVING** Klausel gefiltert werden
- Syntax:
... **GROUP BY** <Kriterien...> **HAVING** <Filter Bedingung>
- Gruppenfilterung mittels **HAVING** analog zu **WHERE** Klausel für einzelne Tupel
[Beispiel ../Code/275_SELECT_GROUP_BY_HAVING.sql](#)

Ablaufplan SELECT Statement

- bekannte Syntax (vereinfacht):

```
SELECT [DISTINCT] select_expr [, select_expr ...]  
FROM table_references  
[WHERE where_condition]  
[GROUP BY <spaltenname1> [, <spaltenname2> ...]]  
[HAVING where_condition]  
[ORDER BY <spaltenname1> [ASC | DESC] [,  
↪ <spaltenname2> [ASC | DESC] ...]]  
[LIMIT row_count];
```

- Abarbeitung durch das DBMS erfolgt gemäß der Reihenfolge der einzelnen Klauseln des **SELECT** Statements

9 SQL - Data Query Language - Teil 2

- Sortieren und Limitieren der Ergebnismenge
- Gruppierung und Aggregatfunktionen
- Verknüpfung mehrerer Relationen mittels Join

- **Normalisierung** verhindert Anomalien durch **Vermeidung von Redundanz** im Datenbestand
- aber: Tupel werden ggf. auf mehrere Relationen **verteilt**
- Verknüpfung mittels **Fremdschlüsselbeziehungen** der Relationen im Schema
- mittels **JOIN** lassen sich mehrere Relationen verknüpfen
- **Verknüpfte Tupel** stellen Datenquelle für Select Expressions dar
Beispiel [../Code/280_SELECT_JOIN.sql](#)

- **INNER JOIN** → nur Tupel, die in **beiden Relationen enthalten** sind, werden als Join Ergebnis verknüpft
- **OUTER JOIN** → Tupel, die nicht in beiden Relationen enthalten sind, werden ebenfalls ins Join Ergebnis übernommen
 - **LEFT OUTER JOIN** → Tupel, die lediglich in der „linken“ Relation existieren, werden übernommen
 - **RIGHT OUTER JOIN** → Tupel, die lediglich in der „rechten“ Relation existieren, werden übernommen
 - manche DBMS unterstützen zusätzlich:
FULL OUTER JOIN → alle Tupel aus der „linken“ und „rechten“ Relation werden übernommen

- **Standardfall** eines Joins

- allgemeine Syntax:

```
SELECT ... FROM <relation1>  
[INNER] JOIN <relation2> ON <Identitätskriterium>;
```

- häufig eingesetztes Identitätskriterium ist der Vergleich zweier Attribute:

```
SELECT ... FROM <relation1>  
[INNER] JOIN <relation2> ON <relation1>.<Attribut1> =  
    ↪ <relation2>.<Attribut2>;
```

- kombiniert die Attribute der Tupel aus beiden Relationen
- Im Ergebnis des Joins sind nur Tupel enthalten, die dem Identitätskriterium entsprechen

Beispiel [../Code/281_SELECT_INNER_JOIN.sql](#)

Ausgangssituation:

- zwei Relationen **rel1** und **rel2**
- Fremdschlüssel **rel2.FK** referenziert Primärschlüssel **rel1.PK**

rel1:

PK	Attr
1	A1
2	A2
3	A3

rel2:

PK	FK	Attr
1	1	B1
2	1	B2
3	3	B3

- Ziel: Ermittlung des **INNER JOIN** Ergebnisses der beiden Relationen
SELECT * FROM rel1 JOIN rel2 ON rel1.PK = rel2.FK
- Hinweis: die nachfolgenden Folien dienen nur zur Visualisierung, die tatsächlichen Abläufe im DBMS sind effizienter

Schritt 1: Kreuzprodukt der beiden Relationen

- **alle Tupel** der Relation **rel1** werden mit **allen Tupeln** der Relation **rel2** kombiniert

rel1.PK	rel1.Attr	rel2.PK	rel2.FK	rel2.Attr
1	A1	1	1	B1
1	A1	2	1	B2
1	A1	3	3	B3
2	A2	1	1	B1
2	A2	2	1	B2
2	A2	3	3	B3
3	A3	1	1	B1
3	A3	2	1	B2
3	A3	3	3	B3

Schritt 2: Streiche alle Tupel, für die das Identitätskriterium **nicht** erfüllt ist:

rel1.PK	rel1.Attr	rel2.PK	rel2.FK	rel2.Attr
1	A1	1	1	B1
1	A1	2	1	B2
1	A1	3	3	B3
2	A2	1	1	B1
2	A2	2	1	B2
2	A2	3	3	B3
3	A3	1	1	B1
3	A3	2	1	B2
3	A3	3	3	B3

Outer Joins

- **OUTER** muss explizit angegeben werden

- allgemeine Syntax:

```
SELECT ... FROM <relation1>  
[LEFT | RIGHT] OUTER JOIN <relation2> ON  
    ↪ <Identitätskriterium>;
```

- **LEFT** oder **RIGHT** müssen angegeben werden
- **LEFT** bezieht sich auf die Tupel **vor** dem Schlüsselwort **JOIN** (hier: <relation1>)
- **RIGHT** bezieht sich auf die Tupel **nach** dem Schlüsselwort **JOIN** (hier: <relation2>)

Beispiel [../Code/282_SELECT_OUTER_JOIN.sql](#)

Visualisierung der Outer Joins

Ausgangssituation (identisch zur Visualisierung des Inner Joins):

- zwei Relationen **rel1** und **rel2**
- Fremdschlüssel **rel2.FK** referenziert Primärschlüssel **rel1.PK**

rel1:

PK	Attr
1	A1
2	A2
3	A3

rel2:

PK	FK	Attr
1	1	B1
2	1	B2
3	3	B3

- Ziel: Ermittlung des **LEFT OUTER JOIN** Ergebnisses der beiden Relationen

```
SELECT * FROM rel1 LEFT OUTER JOIN rel2 ON rel1.PK =  
↪ rel2.FK
```

→ Hinweis: die nachfolgenden Folien dienen nur zur Visualisierung, die tatsächlichen Abläufe im DBMS sind effizienter

Visualisierung der Outer Joins

Schritt 1: Kreuzprodukt der beiden Relationen

Schritt 2: Streiche alle Tupel, für die das Identitätskriterium **nicht erfüllt** ist:

rel1.PK	rel1.Attr	rel2.PK	rel2.FK	rel2.Attr
1	A1	1	1	B1
1	A1	2	1	B2
3	A3	3	3	B3

Schritt 3: Ergänze alle Tupel aus **rel1**, die bisher nicht im Ergebnis enthalten sind. Fehlende für Attribute von **rel2** werden mit **NULL** ergänzt:

rel1.PK	rel1.Attr	rel2.PK	rel2.FK	rel2.Attr
1	A1	1	1	B1
1	A1	2	1	B2
3	A3	3	3	B3
2	A2	NULL	NULL	NULL

- Stimmen die **Namen** der zu vergleichenden Attribute **überein**, kann folgende Kurzschreibweise verwendet werden:

```
SELECT ... FROM <relation1>  
[...] JOIN <relation2> USING(<Name des Attributs>);
```

- Hinweis: im Gegensatz zur Verwendung von **ON** wird bei **USING** das Attribut, das in beiden Relationen enthalten ist **nur einmal** ausgegeben

Beispiel [../Code/288_SELECT_JOIN_USING.sql](#)

- sollen **mehr als zwei** Relationen verknüpft werden, spricht man von **verketteten Joins**
 - Joins werden **der Reihe nach** abgearbeitet
 - **LEFT** bei späteren Joins bezieht sich auf **das Ergebnis der vorherigen Joins**
 - Join Arten können **gemischt** werden
- Beispiel [../Code/289_SELECT_verkettete_Joins.sql](#)

- **SELECT** Ergebnisse können **sortiert** und in der Anzahl **limitiert** werden
- **SELECT** Ergebnisse können **gruppiert** werden
- **Aggregatfunktionen** dienen zum Zusammenfassen der Attributwerte, die nicht zur Gruppierung verwendet wurden
- Erzeugte Gruppen können mittels **HAVING** **gefiltert** werden
- Verknüpfungen **mehrerer Relationen** werden als **JOIN** bezeichnet
- Unterscheidung von **INNER** und **OUTER** Joins
- Verknüpfung von mehr als zwei Relationen durch **verkettete Joins** möglich

- 8 SQL - Data Query Language
- 9 SQL - Data Query Language - Teil 2
- 10 SQL - Mengenoperationen und Views**

Wiederholung

10 SQL - Mengenoperationen und Views

- Mengenoperationen
- Views

- ergänzend zu **JOIN**: **Mengenoperatoren** zur Verknüpfung mehrerer Datenmengen
- verfügbare Mengenoperatoren (abhängig vom DBMS):
 - **UNION**: Vereinigungsmenge
 - **INTERSECT**: Schnittmenge
 - **EXCEPT**: Differenzmenge
- Filtern oder Beibehalten von **Duplikaten** möglich
- **Sortieren** und **Limitieren** der Ergebnismenge

Wichtig

Die Attribute der beiden Teilmengen müssen in **Anzahl und Datentypen** übereinstimmen!

Join

- kombiniert Attribute mehrerer Tupel
- Die Anzahl an Attributen pro Tupel **nimmt zu**
- „**horizontale**“ Integration
- Stellt Datenquelle für **SELECT** Statement bereit
- wirkt „vor“ dem **SELECT**

Mengenoperationen

- kombinieren Tupel mit **identischen** Attributen
- Die Anzahl an Attributen pro Tupel **bleibt unverändert**
- „**vertikale**“ Integration
- Verarbeiten Ergebnis von **SELECT** Statements
- wirken „nach“ dem **SELECT**

Vereinigungsmenge

- **fasst** Tupel aus mehreren Ergebnismengen **zusammen**

- Syntax:

```
SELECT ...  
UNION [ALL | DISTINCT] SELECT ...  
[ UNION [ALL | DISTINCT] SELECT ... ]  
[ ORDER BY column [, column ...]]  
[ LIMIT [offset ,] rowcount];
```

- Die **Spaltenüberschriften** der Vereinigungsmenge werden vom **ersten SELECT** Statement übernommen
- Umgang mit **Duplikaten**:
 - **DISTINCT** (Standard) → Filtert Duplikate aus der Vereinigungsmenge
 - **ALL** → belässt Duplikate in der Vereinigungsmenge
- **Achtung**: die einzelnen **SELECT** Statements sind **nicht eigenständig**
→ „;“ erst ganz am Ende!

Beispiel [../Code/300_UNION.sql](#)

- liefert alle Tupel, die **in beiden Teilmengen** vorhanden sind

- Syntax:

```
SELECT ...  
INTERSECT [ALL | DISTINCT] SELECT ...  
[ UNION [ALL | DISTINCT] SELECT ... ]  
[ ORDER BY column [, column ...]]  
[ LIMIT [offset ,] rowcount];
```

- Regeln zu **Spaltenüberschriften** und **Duplikatfilterung** wie bei **UNION**

Beispiel [../Code/305_INTERSECT.sql](#)

- liefert Tupel, die **nur in der ersten Teilmenge** vorhanden sind
- Tupel, die in beiden Mengen enthalten sind, werden **ausgefiltert**

- Syntax:

```
SELECT ...  
EXCEPT [ALL | DISTINCT] SELECT ...  
[ UNION [ALL | DISTINCT] SELECT ... ]  
[ ORDER BY column [, column ...]]  
[ LIMIT [offset ,] rowcount];
```

- Regeln zu **Spaltenüberschriften** und **Duplikatfilterung** wie bei **UNION**

Beispiel [../Code/310_EXCEPT.sql](#)

- Ergebnis einer Mengenoperation kann **als Teilmenge** für eine **folgende Mengenoperation** verwendet werden
→ **Verkettung** von Mengenoperationen
- Regeln zur **Abarbeitungsreihenfolge**:
 - **INTERSECT** hat **höchste Priorität** → wird zuerst ausgeführt
 - **UNION** und **EXCEPT** haben **niedrigere aber gleiche Priorität** → werden der Reihe nach ausgeführt
- **Beeinflussung** der Abarbeitungsreihenfolge mittels **Klammern** möglich

- **Sortieren** mittels **ORDER BY** ...
- **Limitieren** mittels **LIMIT** ...
- Syntax und Regeln zum Sortieren und Limitieren identisch zu „einfachem“ **SELECT** Statement

Beispiel

```
../Code/312_Sortieren_und_Limitieren_bei_Mengenoperationen.sql
```

- **Achtung:** sollen die Ergebnisse der einzelnen **SELECT** Statements limitiert werden, **müssen** die **SELECT** Statements geklammert werden!

Beispiel

```
../Code/313_Limitieren_des_letzten_SELECTs_bei_Mengenoperationen.sql
```

Weiterverarbeitung von Mengenoperationen

- Das **Ergebnis** (verketteter) Mengenoperationen kann **als Datenquelle** für weitere **SELECT** Statements verwendet werden
- Vorgehen:
 - **Zusammenfassen** der Mengenoperationen durch **Klammern**
 - mittels **AS** <tabellenalias> erhält das Mengenoperationsergebnis einen eigenen Namen
→ Referenzierung z.B. in **JOIN** möglich
 - Einbettung in **FROM** Klausel des übergeordneten **SELECT** Statements
- grundlegende Syntax:

```
SELECT ...  
FROM ( SELECT ...  
      [UNION | INTERSECT | EXCEPT]  
      SELECT ...  
    ) AS <tabellenalias>  
...
```

Beispiel [../Code/315_Subquery_mit_Mengenoperation.sql](#)

10 SQL - Mengenoperationen und Views

- Mengenoperationen
- Views

- **VIEWs** dienen zur Erstellung **benutzerdefinierter Sichten** auf die Inhalte eines Schemas
- Views enthalten **keine eigenen Daten**
- **Anwendungsfälle:**
 - Hinterlegen häufig verwendeter, komplexer **SELECT** Statements **als Teil des Schemas**
 - Implementierung **abgeleiteter Attribute** des Entity Relationship Modells
 - Bereitstellung von aufbereiteten Daten für **unterschiedliche Benutzergruppen**
 - **Einschränkung der Zugriffsrechte** auf definierte Informationen
 - **Performance Optimierung** mittels **Materialized Views** (in MariaDB leider nicht vorhanden)
- **Grundkonzept:**
 - **SELECT** Statement als **Ausgangsbasis**
 - hinterlegen dieses **SELECT** Statements mittels DDL Methoden **als Teil des Schemas**

- Kombination aus Data **Definition** Language und Data **Query** Language
- Syntax (vereinfacht):

```
CREATE [OR REPLACE] VIEW [IF NOT EXISTS] <viewname>  
AS <selectStatement>
```

Achtung

strukturelle Änderungen an Relation(en), auf die sich eine View bezieht, können die View **ungültig** machen!

Views am Beispiel: „Vorlesungsverzeichnis“

- Ziel: Bereitstellung eines Vorlesungsverzeichnisses mit Daten zu Dozenten und Vorlesungen
 - enthaltene Informationen:
 - Personalnummer (aus Dozenten- **und** Vorlesungsrelation)
 - Name des Dozenten
 - E-Mail Adresse des Dozenten
 - Modulnummer
 - Name des Moduls
 - Schritt 1: **SELECT** Statement erstellen
 - Schritt 2: **VIEW** basierend auf **SELECT** Statement erstellen
- Beispiel `../Code/320_CREATE_VIEW_Vorlesungsverzeichnis.sql`

Views am Beispiel: „Notenblatt“

- Ziel: Bereitstellung einer Notenübersicht als View „Notenblatt“
 - enthaltene Informationen:
 - Matrikelnummer
 - Name
 - Modulnummer
 - beste bisher erreichte Note im jeweiligen Modul
 - Schritt 1: **SELECT** Statement erstellen
 - Schritt 2: **VIEW** basierend auf **SELECT** Statement erstellen
- Beispiel [../Code/321_CREATE_VIEW_Notenblatt.sql](#)

- die Datenquelle des **SELECT** Statements einer View kann **selbst wiederum Views enthalten**
- Beispiel: Übersicht aktueller Notenschnitt nach Matrikelnummer
 - View „Notenblatt“ als Ausgangsbasis
 - **Gruppierung** der Tupel nach Matrikelnummer
 - **Durchschnittsberechnung** der Noten pro Gruppe mittels **AVG()** Aggregatfunktion

Beispiel [../Code/322_CREATE_VIEW_Notenschnitt.sql](#)

- Views lassen sich in **SELECT** Statements ohne Einschränkungen **wie Relationen** verwenden
- DBMS interne Ausführung von **SELECT ... FROM <view>** in **zwei Schritten**:
 - Ausführen des **als View hinterlegten SELECT** Statements
 - Ausführen des „äußeren“ **SELECT** Statements

Beispiel [../Code/323_SELECT_FROM_VIEW.sql](#)

- Views können **unter bestimmten Voraussetzungen** zum **Aktualisieren oder Einfügen** von Daten in die zugrundeliegenden Relationen verwendet werden
- Voraussetzungen (Auszug):
 - **keine Duplikatfilterung** mittels **DISTINCT** im **SELECT** Statement
 - **keine Subqueries** in den Select Expression bzw. der **WHERE** Klausel
 - **keine Gruppierung und Aggregatfunktionen** (**GROUP BY**, **HAVING**, **MAX**, etc.)
 - **kein Outer Join** im **SELECT** Statement
 - **keine Vereinigungsmenge** (**UNION [ALL]**)
 - **keine Beschränkung der Anzahl** mittels **LIMIT**
 - nur Attribute **einer** zugrundeliegenden Relation pro **INSERT** bzw. **UPDATE** Statement
 - View enthält **alle nicht optionalen** (z.B. mit **NOT NULL** definierten) Attribute der zugrundeliegenden Relation

Schreibender Zugriff auf Views am Beispiel

- Prüfung der „Updatefähigkeit“ mittels

```
SELECT TABLE_NAME, IS_UPDATABLE FROM
```

```
  ↳ INFORMATION_SCHEMA.VIEWS;
```

- Views „Notenblatt“ und „Notenschnitt“ sind **nicht Update-fähig**, da Gruppierung verwendet wird
- View „Vorlesungsverzeichnis“ ist **Update-fähig**

Beispiel [../Code/324_INSERT_INTO_UPDATE_VIA_VIEW.sql](#)

- Verknüpfung mehrerer Datenmengen mittels **Mengenoperatoren**
 - Vereinigungsmenge: **UNION**
 - Schnittmenge: **INTERSECT**
 - Differenzmenge: **EXCEPT**
 - **Verkettung** mehrerer Mengenoperationen möglich
- Erstellung **benutzerdefinierter Sichten** mittels VIEWS
 - **SELECT** als Grundlage
 - enthalten **keine eigenen Daten**
 - **Verschachtelung** möglich
 - **uneingeschränkter lesender** Zugriff möglich
 - **eingeschränkter schreibender** Zugriff „durch“ die View auf die eigentliche Relation

nächstes Mal: Fragestunde

**Fragen gerne bereits im Vorfeld per
Mail, Moodle, ...**